

UNIVERSIDAD POLITÉCNICA DE MADRID
Escuela Técnica Superior de Ingeniería de Sistemas
Informáticos



UNIVERSIDAD
POLITÉCNICA
DE MADRID



Máster
Deep Learning
Universidad Politécnica de Madrid

Trabajo Fin de Máster

Sistema LLM para análisis financiero histórico
mediante generación automática de código

MÁSTER DE FORMACIÓN PERMANENTE EN DEEP LEARNING

Presentado por:

Carlos Ruiz Oyarzun

Bajo la dirección de:

Adrián Girón Jiménez

Alejandro Delgado Peribáñez

Madrid, 2026

Resumen

Este Trabajo Fin de Máster propone el desarrollo incremental de un sistema orientado al análisis financiero histórico con modelos de lenguaje. El sistema parte de una consulta de usuario, datos de mercado almacenados en CSV y metadatos como tickers, fechas o intervalo temporal. A partir de esa entrada, el prototipo valida la solicitud, utiliza un LLM para interpretar la intención, genera código Python ejecutable, valida dicho código, lo lanza en un proceso controlado y transforma la salida estructurada en una respuesta comprensible mediante clientes compatibles con la interfaz Chat Completions.

El MVP actual soporta varias familias de consultas, desde análisis simples de crecimiento hasta comparativas con tablas, datos de visualización y explicaciones para usuario final. La memoria separa el papel de dos agentes funcionales: un primer bloque LLM que planifica el análisis y genera código, y un segundo bloque que interpreta resultados ya calculados. La evaluación combina pruebas funcionales, escenarios de error controlado, ejemplos A/B/C y comparación exploratoria entre proveedores LLM. Los resultados muestran que el sistema puede mantener el cálculo financiero bajo control programático, conservar trazabilidad sobre los artefactos y limitar la respuesta al análisis histórico, sin predicción futura ni recomendación de inversión.

Palabras clave: análisis financiero, modelos de lenguaje, generación de código, ejecución controlada, clientes LLM compatibles.

Abstract

This Master's Thesis proposes the incremental development of a language-model-driven system for historical financial analysis. The system starts from a user query, market data stored in CSV files and metadata such as tickers, dates or time interval. From this input, the prototype validates the request, uses an LLM to interpret the user's intent, generates executable Python code, validates that code, runs it in a controlled process and transforms the structured output into an interpretable response through Chat Completions-compatible clients.

The current MVP supports several families of queries, ranging from simple growth analysis to comparisons with tables, visualization data and final-user explanations. The report separates the role of two functional agents: a first LLM block that plans the analysis and generates code, and a second block that interprets already computed results. The evaluation combines functional tests, controlled error scenarios, A/B/C examples and an exploratory comparison across LLM providers. The results show that the system can keep financial computation under programmatic control, preserve traceability over generated artifacts and limit the response to historical analysis, without future prediction or investment recommendation.

Keywords: financial analysis, language models, code generation, controlled execution, compatible LLM clients.

Índice general

Resumen	I
Abstract	II
1. Introducción	1
1.1. Contexto y definición del problema	1
1.2. Motivación	2
1.3. Objetivos	3
1.4. Alcance	4
1.5. Estructura de la memoria	4
2. Estado de la cuestión	5
2.1. Enfoque de la revisión	5
2.2. Modelos de lenguaje en finanzas	5
2.3. Agentes LLM aplicados a finanzas	6
2.4. LLM con herramientas y ejecución	6
2.5. Generación de código para análisis de datos	7
2.6. Modelos multimodales y salidas visuales	7
2.7. Fiabilidad, alucinaciones y trazabilidad	8
2.8. Comparativa y posicionamiento de la propuesta	9
3. Datos y material de trabajo	10
3.1. Fuente de datos y datos obtenidos	10
3.2. Catálogo de consultas y proceso de descarga	11
3.3. Estructura de los CSV	13
3.4. Análisis exploratorio de datos	14
3.5. Calidad, cobertura y limitaciones	15
3.6. Material generado por el flujo y trazabilidad	15
4. Metodología	18
4.1. Enfoque de desarrollo	18
4.2. Evolución incremental del MVP	19

4.3.	Criterios de validación	20
4.4.	Criterios cualitativos de calidad del análisis	22
4.5.	Niveles de profundidad del análisis	23
4.6.	Evaluación con proveedores LLM	23
4.7.	Revisión humana	24
5.	Diseño del sistema	25
5.1.	Arquitectura general	25
5.2.	Nodos del flujo	27
5.3.	Contrato del código generado	28
5.4.	Gestión de errores	29
6.	Implementación	30
6.1.	Estructura del código	30
6.2.	Repositorio y organización del trabajo	30
6.3.	Contratos de datos	31
6.4.	Pipeline LLM	32
6.5.	Control de seguridad	33
6.6.	Reparación y compactación	33
6.7.	Pruebas	34
7.	Integración de modelos de lenguaje	35
7.1.	Papel del LLM en el sistema	35
7.2.	Separación entre agentes y prompts	35
7.3.	Pipeline del primer y segundo agente	36
7.4.	Cliente compatible con Chat Completions	37
7.5.	Configuraciones evaluadas	37
7.6.	Gestión de errores de API	38
7.7.	Gestión de credenciales	39
8.	Resultados y evaluación	40
8.1.	Objetivo de la evaluación	40
8.2.	Métricas y criterios	41
8.3.	Niveles cualitativos de salida	42
8.4.	Batería progresiva	43
8.5.	Tipos de salida y ejemplos visuales	44
8.6.	Resultados observados	46
8.7.	Limitaciones metodológicas	46
8.8.	Síntesis	47

9. Aspectos éticos, seguridad y uso responsable	48
9.1. Delimitación financiera del sistema	48
9.2. Riesgos de interpretación	48
9.3. Seguridad en la generación y ejecución de código	48
9.4. Privacidad y datos	49
9.5. Sostenibilidad, costes y recursos	49
9.6. Limitaciones de uso	49
10. Conclusiones y trabajo futuro	50
Referencias	52
A. Anexos	56
A.1. Ejecución del MVP	56
A.2. Configuración de APIs	57
A.3. Batería de consultas A/B/C	57
A.4. Query de estrés visual	58
A.5. Prompts usados por los agentes	59
A.5.1. Prompt base del sistema	59
A.5.2. Agente 1: interpretación de consulta	59
A.5.3. Agente 1: generación de código	59
A.5.4. Reparación de código	60
A.5.5. Agente 2: interpretación final	60
A.6. Ejecuciones completas resumidas por nivel	60
A.6.1. Fragmentos de código generado	61
A.6.2. Artefactos de trazabilidad	62
A.7. Documentación estable	62
A.8. Plantilla de revisión manual	63
A.9. Documentación interna de apoyo	65

Índice de tablas

2.1. Comparativa entre trabajos relacionados y la propuesta del TFM.	9
3.1. Datos obtenidos y finalidad de cada conjunto dentro del MVP.	11
3.2. Estructura semántica de una consulta financiera antes de su procesamiento.	13
3.3. Campos OHLCV presentes en los CSV de mercado.	13
3.4. Resumen de cobertura y calidad del conjunto de datos.	14
3.5. Muestra de cobertura temporal obtenida en el EDA.	14
3.6. Métricas exploratorias por serie utilizadas para contextualizar los casos de prueba.	15
3.7. Material generado por el flujo y papel de cada agente.	16
4.1. Criterios de validación aplicados al flujo del prototipo.	21
5.1. Responsabilidades principales de la arquitectura general.	26
6.1. Módulos principales de la implementación.	30
6.2. Contrato operativo del código Python generado por el LLM.	32
6.3. Comprobaciones principales del validador de seguridad.	33
7.1. Diferencias entre los prompts usados en cada bloque LLM.	36
8.1. Dimensiones usadas para valorar la calidad técnica y analítica de cada ejecución.	42
8.2. Niveles cualitativos de profundidad del análisis.	43
8.3. Tipos de salida generados por el workflow y uso dentro de la evaluación.	44
8.4. Completitud observada por configuración y nivel en la batería de 15 consultas.	46
A.1. Batería cualitativa de 15 consultas por nivel de profundidad.	58
A.2. Resumen de ejecuciones A/B/C con <code>openai/gpt-oss-20b</code>	61
A.3. Artefactos reales asociados a los ejemplos de ejecución A/B/C.	62
A.4. Plantilla de revisión manual cualitativa.	64
A.5. Relación entre documentación Markdown y secciones de la memoria.	65

Índice de figuras

1.1. Mapa conceptual de relaciones entre inteligencia artificial, aprendizaje automático, redes neuronales, aprendizaje profundo e inteligencia artificial generativa. Fuente: Analytics Vidhya [2].	2
5.1. Arquitectura general del sistema desde la consulta del usuario hasta la respuesta trazable.	26
8.1. Ejemplo de salida visual: evolución normalizada base 100 de QQQ y SPY en 2024.	45
8.2. Ejemplo de salida visual: drawdown histórico de QQQ y SPY en 2024. .	45
8.3. Completitud por modelo y nivel en la pasada exploratoria de 15 consultas.	47

Capítulo 1

Introducción

1.1. Contexto y definición del problema

La inteligencia artificial ha evolucionado desde sistemas basados en reglas y técnicas estadísticas hasta modelos capaces de aprender representaciones complejas a partir de grandes volúmenes de datos. Dentro de esta evolución, el aprendizaje automático permitió entrenar modelos a partir de ejemplos; el aprendizaje profundo amplió esa capacidad mediante redes neuronales con muchas capas; y la inteligencia artificial generativa incorporó modelos capaces de producir texto, código, imágenes u otros artefactos a partir de instrucciones. Un punto de inflexión especialmente relevante fue la arquitectura Transformer, introducida en el trabajo *Attention Is All You Need* [1]. Su mecanismo de atención facilitó el entrenamiento de modelos de lenguaje de gran escala y se convirtió en una de las bases técnicas sobre las que se apoyan los LLM actuales.

Este TFM se sitúa dentro de esa evolución, pero no desde la perspectiva de entrenar un nuevo modelo. El foco está en integrar modelos de lenguaje ya disponibles dentro de un sistema software verificable. En concreto, el proyecto explora cómo un LLM puede ayudar a interpretar consultas financieras, generar código Python e interpretar resultados, manteniendo el cálculo bajo control programático y conservando artefactos auditables.

El análisis financiero histórico combina carga de datos, validación de entradas, selección del tipo de análisis, cálculo de métricas, generación de código, interpretación de resultados y comunicación final al usuario. En muchos prototipos basados en modelos de lenguaje estas fases se mezclan dentro de una única llamada generativa, lo que dificulta la trazabilidad del resultado y aumenta el riesgo de errores silenciosos.

Este trabajo plantea una aproximación controlada: el sistema parte de una entrada estructurada con consulta, tickers, fechas y rutas CSV; valida los datos mínimos; utiliza un LLM central para interpretar la intención; genera código Python; valida el contrato de seguridad del script; ejecuta el código en un entorno controlado; y finalmente usa el

LLM para explicar las métricas obtenidas.

Para situar el alcance tecnológico del trabajo, la Figura 1.1 resume la relación entre inteligencia artificial, aprendizaje automático, aprendizaje profundo e inteligencia artificial generativa. El TFM no se centra en entrenar modelos ni en desarrollar nuevas arquitecturas neuronales, sino en integrar modelos de lenguaje dentro de un sistema trazable de análisis financiero histórico.

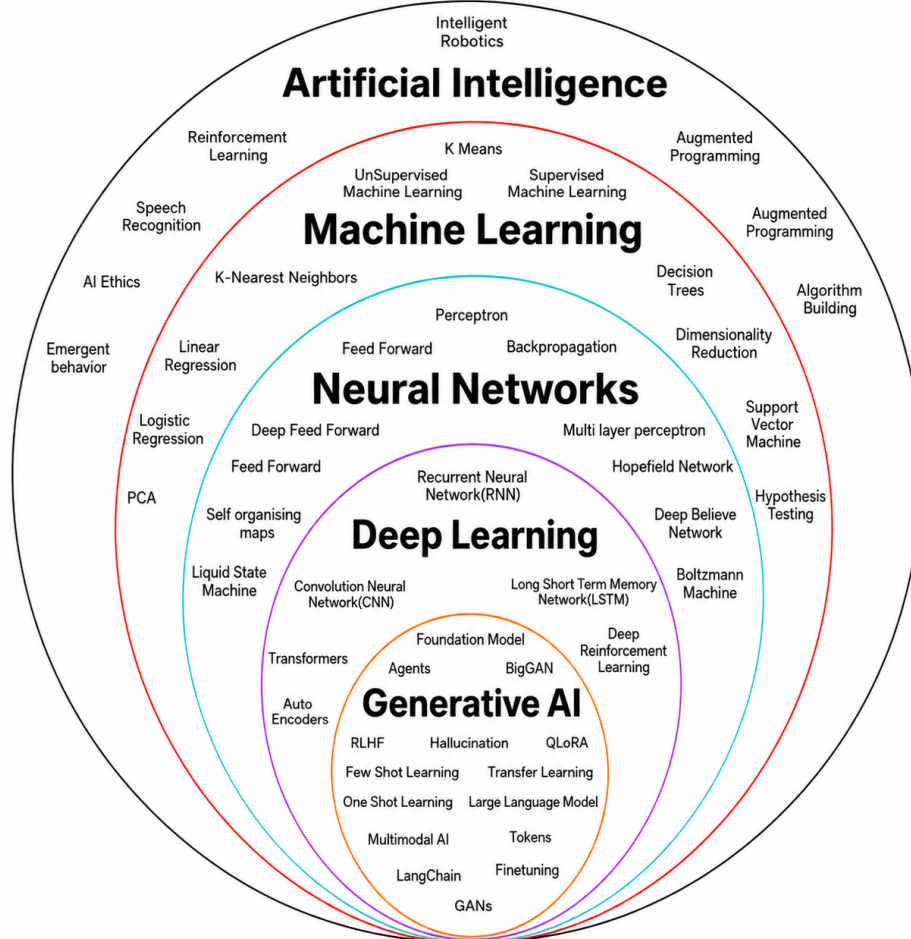


Figura 1.1: Mapa conceptual de relaciones entre inteligencia artificial, aprendizaje automático, redes neuronales, aprendizaje profundo e inteligencia artificial generativa. Fuente: Analytics Vidhya [2].

1.2. Motivación

La motivación principal del proyecto es explorar cómo un flujo LLM con ejecución controlada puede aportar orden, modularidad y robustez a un sistema de análisis financiero asistido por inteligencia artificial. La aplicación de modelos de lenguaje al dominio financiero resulta atractiva por su capacidad para interpretar peticiones, generar código y adaptar el lenguaje al usuario, pero también exige mecanismos de control, especialmente cuando la respuesta final puede confundirse con una recomendación fi-

nanciera.

Además, el proyecto nace de un interés personal por la intersección entre inteligencia artificial, programación y análisis financiero. La disponibilidad de modelos generativos abre la posibilidad de construir prototipos útiles con una rapidez que hace pocos años era difícil de imaginar. Sin embargo, precisamente por esa facilidad, resulta importante defender un enfoque responsable: las salidas del modelo deben revisarse, contrastarse con datos y situarse dentro de los límites del conocimiento experto. En este TFM, esa idea se traduce en una arquitectura donde el LLM ayuda, pero no sustituye la validación, la ejecución controlada ni la revisión crítica de resultados.

El interés del trabajo no está en construir un producto financiero completo, sino en estudiar si un prototipo incremental puede convertir consultas históricas en cálculos trazables y explicaciones prudentes. Por ello se evita que el LLM actúe como una caja negra que responde directamente al usuario: el modelo interpreta la intencionalidad de la consulta y genera código, pero el sistema valida dicho código antes de ejecutarlo y captura los resultados en formato estructurado. Después, el LLM redacta la respuesta final a partir de métricas ya calculadas. Si el proveedor no está configurado o falla, el sistema informa el error y no genera una respuesta alternativa sin modelo.

En esta memoria se utiliza el término MVP como *prototipo mínimo viable* dentro de una evolución por etapas, no como producto comercial cerrado. El primer MVP validó dos intencionalidades básicas, crecimiento y comparación; el segundo amplió la cobertura a seis tipos de análisis; y la versión actual incorpora generación de código controlada, validación de seguridad, ejecución aislada, artefactos auditables y comparación entre proveedores LLM. Esta evolución se detalla en la Sección 4.2.

1.3. Objetivos

El objetivo general del trabajo es desarrollar y validar un prototipo basado en LLM capaz de ejecutar análisis financieros históricos a partir de entradas estructuradas y datos en CSV.

Los objetivos específicos son:

- Definir un flujo que separe validación, interpretación de intención, generación de código, control de seguridad, ejecución e interpretación final.
- Diseñar un primer agente funcional capaz de interpretar la consulta, identificar la profundidad esperada del análisis y construir un plan estructurado.
- Generar código Python a partir del plan analítico y del mensaje original del usuario.

- Validar el código generado mediante contrato de script, importaciones permitidas y salida JSON.
- Ejecutar el código en un entorno controlado y conservar artefactos auditables.
- Diseñar un segundo agente funcional que redacte la respuesta final a partir de métricas ya calculadas, sin añadir datos externos ni recomendaciones de inversión.
- Cubrir consultas de distinta profundidad, desde respuestas básicas hasta salidas con tablas, datos visuales y comparativas detalladas.
- Evaluar el MVP mediante casos funcionales, escenarios de error controlado, ejemplos A/B/C y comparación exploratoria entre proveedores LLM.

1.4. Alcance

El sistema se limita al análisis histórico-descriptivo de activos financieros a partir de datos ya descargados. No realiza predicción futura, recomendación de inversión ni análisis fundamental complejo. Esta restricción es deliberada: el foco del TFM no es construir un asesor financiero, sino estudiar una arquitectura LLM trazable para análisis histórico con generación y ejecución controlada de código.

1.5. Estructura de la memoria

La memoria se organiza siguiendo la lógica de desarrollo del proyecto. El Capítulo 2 presenta el estado de la cuestión, con especial atención a modelos de lenguaje en finanzas, uso de herramientas externas, generación de código, multimodalidad y trazabilidad. El Capítulo 3 describe los datos utilizados, su procedencia, su organización en CSV y los artefactos generados por el flujo. El Capítulo 4 expone la metodología incremental seguida durante el desarrollo del MVP.

Los capítulos centrales describen la arquitectura y la implementación. El Capítulo 5 presenta el diseño general del sistema y la figura de flujo orientada a usuario final. El Capítulo 6 detalla los módulos de código, el contrato de ejecución, el control de seguridad y la compactación de salidas. El Capítulo 7 explica la integración de modelos de lenguaje, la separación entre agentes, los prompts y los perfiles evaluados. El Capítulo 8 recoge el planteamiento de evaluación, los niveles A/B/C y ejemplos visuales de salida. Finalmente, el Capítulo 9 aborda consideraciones éticas y de seguridad, el Capítulo 10 resume conclusiones y trabajo futuro, y los anexos incluyen configuraciones, batería de consultas, prompts y ejecuciones completas resumidas.

Capítulo 2

Estado de la cuestión

2.1. Enfoque de la revisión

El objetivo de este capítulo es situar el TFM dentro del contexto actual de los modelos de lenguaje aplicados al análisis financiero y a la automatización de tareas de datos. La revisión no pretende cubrir todo el campo de la inteligencia artificial financiera, sino seleccionar aquellas líneas de trabajo que ayudan a justificar las decisiones del proyecto: uso de LLM en finanzas, agentes con herramientas externas, generación de código, salidas visuales, fiabilidad y trazabilidad.

Esta delimitación es importante porque el sistema desarrollado no consiste en entrenar un nuevo modelo financiero ni en construir un asesor de inversión. La propuesta se centra en una arquitectura de integración: una consulta de usuario se valida, se interpreta mediante LLM, se transforma en código Python, se revisa antes de ejecutarse y se convierte en una respuesta final basada en resultados ya calculados. Por tanto, el estado de la cuestión se orienta a comparar el TFM con trabajos que combinan lenguaje natural, dominio financiero, herramientas, código y control de ejecución.

2.2. Modelos de lenguaje en finanzas

El uso de procesamiento del lenguaje natural en finanzas es anterior a los modelos generativos actuales. FinBERT adapta modelos tipo BERT al dominio financiero y muestra que el lenguaje de mercado tiene particularidades suficientes como para justificar modelos especializados, especialmente en tareas de sentimiento financiero [3]. Con la aparición de LLM de mayor escala, BloombergGPT y FinGPT ampliaron el foco hacia modelos entrenados o adaptados con datos financieros, documentación corporativa, noticias, series de mercado y otros recursos específicos del sector [4], [5].

Estos trabajos son relevantes porque demuestran que el dominio financiero se beneficia de modelos capaces de comprender terminología, contexto y relaciones propias del

sector. Sin embargo, se sitúan en una escala distinta a la de este TFM. BloombergGPT y FinGPT se centran en crear o adaptar modelos; este proyecto, en cambio, integra modelos disponibles mediante un cliente común y estudia cómo encajarlos en un flujo verificable de análisis histórico.

La diferencia también afecta al alcance funcional. El sistema de esta memoria no analiza noticias, documentos corporativos ni sentimiento de mercado, y tampoco intenta predecir precios. Trabaja con datos históricos locales en CSV y calcula métricas descriptivas. Por ello, la aportación no se encuentra en el entrenamiento del modelo, sino en el diseño de un proceso que separa interpretación, cálculo, validación y explicación.

2.3. Agentes LLM aplicados a finanzas

En trabajos recientes aparecen sistemas financieros con componentes más próximos a agentes. FinRobot propone un agente de investigación y valoración de renta variable apoyado en LLM [6]. TradingAgents plantea un marco multiagente orientado a tareas de trading, donde distintos agentes simulan roles de análisis, debate o decisión [7]. Estos enfoques reflejan una tendencia clara: los LLM dejan de actuar únicamente como generadores de texto y pasan a formar parte de sistemas con roles, herramientas, memoria de trabajo y salidas especializadas.

La relación con este TFM es directa, pero también conviene marcar la diferencia. El prototipo desarrollado no busca tomar decisiones de trading ni realizar valoración fundamental completa. Tampoco implementa una sociedad de agentes autónomos que negocien entre sí. La memoria utiliza el término agente en sentido funcional: un primer bloque LLM interpreta la consulta, planifica y genera código; un segundo bloque LLM interpreta resultados calculados para el usuario final. Esta separación permite estudiar la utilidad de los agentes sin desplazar el control del sistema hacia decisiones financieras autónomas.

Desde el punto de vista académico, esta decisión reduce ambición comercial pero aumenta trazabilidad. El interés no es que el modelo actúe libremente sobre el mercado, sino comprobar si un flujo acotado puede convertir lenguaje natural en cálculos reproducibles y explicaciones prudentes.

2.4. LLM con herramientas y ejecución

El diseño técnico del sistema se relaciona con la literatura sobre modelos capaces de razonar y actuar con herramientas externas. ReAct introdujo la combinación de razonamiento y actuación, de modo que el modelo alterna pasos de pensamiento con acciones observables [8]. Toolformer mostró que los modelos pueden aprender a decidir cuándo

llamar a herramientas externas [9]. Las revisiones sobre agentes autónomos basados en LLM describen esta evolución como un paso desde modelos conversacionales hacia sistemas capaces de planificar, actuar y observar resultados en entornos externos [10].

El TFM adopta esta idea general, pero evita que el modelo decida libremente qué herramienta usar o qué acción ejecutar. El flujo está fijado por el propio sistema: validación de entrada, análisis LLM, generación de código, control de seguridad, ejecución e interpretación final. Esta arquitectura limita la autonomía del modelo, pero facilita la depuración y permite revisar cada artefacto intermedio.

En el contexto financiero, esta restricción es especialmente relevante. Una respuesta generada directamente por un LLM puede sonar convincente aunque no esté respaldada por cálculos reales. Al obligar al sistema a pasar por código ejecutado y salida JSON, el prototipo aproxima el comportamiento del modelo a una herramienta de asistencia controlada, no a una fuente opaca de conclusiones.

2.5. Generación de código para análisis de datos

La generación automática de código es una de las capacidades más útiles de los LLM modernos. El trabajo sobre Codex y HumanEval mostró que los modelos entrenados con código pueden sintetizar programas a partir de descripciones en lenguaje natural [11]. En tareas de ciencia de datos, DS-1000 propuso un banco de prueba específico para evaluar generación de código en escenarios realistas con librerías habituales de análisis [12].

Estos trabajos justifican que un sistema de análisis financiero pueda usar un LLM para producir código Python adaptado a la consulta. No obstante, también evidencian un riesgo central: un programa plausible no es necesariamente correcto, seguro ni ejecutable. En este TFM, la generación de código no se trata como una salida final, sino como un artefacto que debe superar controles antes de ejecutarse.

El sistema exige un contrato mínimo: el script debe leer la entrada normalizada, usar librerías permitidas, producir una salida JSON y no realizar acciones fuera del alcance autorizado. Esta capa convierte la generación de código en una fase verificable del workflow. El resultado final no depende solo de la fluidez del modelo, sino de que el código generado pueda ejecutarse y producir métricas coherentes con los datos.

2.6. Modelos multimodales y salidas visuales

Los modelos multimodales amplían los modelos de lenguaje textual para trabajar con más de un tipo de entrada o salida. En lugar de recibir únicamente texto, pueden combinar instrucciones con imágenes, vídeo, audio o documentos, según la arquitectura

y el proveedor. Flamingo es un ejemplo representativo de esta línea, al conectar modelos visuales y lingüísticos para procesar secuencias intercaladas de imágenes, vídeos y texto [13]. En sistemas más recientes, GPT-4V y Gemini describen capacidades para interpretar imágenes, documentos o vídeo junto con instrucciones textuales [14], [15].

Esta línea resulta relevante para el proyecto porque muchas salidas financieras son visuales: curvas normalizadas, drawdowns, tablas comparativas o paneles de evaluación. Un modelo multimodal podría revisar si una gráfica es legible, detectar incoherencias entre una tabla y una imagen o redactar una explicación a partir de una captura del informe.

Sin embargo, el prototipo desarrollado en este TFM no utiliza modelos multimodales dentro del flujo principal. Aunque el proyecto genera PNG para la memoria y para la revisión visual de resultados, el LLM trabaja con texto, planes JSON, código Python y salidas estructuradas. Incluso cuando se evalúa Gemini, que pertenece a una familia con capacidades multimodales, la integración usada aquí es textual: no se envían imágenes al modelo para razonar sobre ellas. Esta aclaración evita sobredimensionar el alcance del sistema y deja la multimodalidad como una línea futura razonable.

2.7. Fiabilidad, alucinaciones y trazabilidad

Una limitación conocida de los modelos generativos es la producción de respuestas plausibles pero no fieles a los datos, fenómeno habitualmente denominado alucinación [16]. En tareas de conocimiento, la generación aumentada con recuperación propuso reducir este problema apoyando la generación en fuentes externas recuperadas explícitamente [17]. Aunque este TFM no implementa un sistema RAG documental, sí adopta una lógica de anclaje: la respuesta final debe apoyarse en datos locales, código ejecutado y resultados estructurados.

En análisis financiero, la trazabilidad no es un detalle técnico secundario. Si una respuesta informa de rentabilidad, volatilidad o drawdown, debe poder revisarse qué datos se usaron, qué cálculo se hizo y qué limitaciones tiene la interpretación. Además, el lenguaje final debe evitar que una descripción histórica se lea como predicción futura o recomendación de inversión.

Por este motivo, la memoria separa cada responsabilidad: los datos se documentan en el capítulo correspondiente, el plan se genera antes del código, el script se valida antes de ejecutarse, la salida se conserva como artefacto y la interpretación final se limita a explicar métricas calculadas. Esta separación es la base metodológica que diferencia el prototipo de una interacción directa con un chatbot generalista.

2.8. Comparativa y posicionamiento de la propuesta

La Tabla 2.1 resume la relación entre las líneas revisadas y el sistema desarrollado en este TFM. La comparación permite identificar qué se toma como inspiración y qué queda fuera del alcance. La propuesta no compite con modelos financieros entrenados a gran escala ni con marcos de trading multiagente; su aportación se sitúa en la integración controlada de LLM, generación de código y análisis histórico reproducible.

Trabajo o línea	Dominio	Enfoque	Relación con este TFM
FinBERT	PLN financiero	Modelo especializado para sentimiento financiero [3].	Muestra la utilidad de adaptar lenguaje al dominio financiero, pero no aborda generación controlada de código ni ejecución de métricas históricas.
BloombergGPT y FinGPT	LLM financieros	Modelos grandes entrenados o adaptados con datos financieros [4], [5].	Se centran en el modelo; este TFM integra modelos ya disponibles dentro de un flujo software verificable.
FinRobot y TradingAgents	Agentes financieros	Agentes LLM para investigación, valoración o marcos multiagente de trading [6], [7].	Inspiran la separación de roles, pero el prototipo no toma decisiones de inversión ni implementa un agente autónomo de trading.
ReAct y Toolformer	LLM con herramientas	Razonamiento combinado con acciones y llamadas a herramientas [8], [9].	Justifican el uso de herramientas externas; el TFM fija un flujo más controlado para mejorar trazabilidad y seguridad.
Codex y DS-1000	Generación de código	Síntesis y evaluación de código funcional o de ciencia de datos [11], [12].	Respaldan la generación de código del MVP y la necesidad de validación estricta antes de ejecutar.
Flamingo, GPT-4V y Gemini	Multimodalidad	Modelos capaces de combinar texto con imágenes, vídeo, audio o documentos [13], [14], [15].	Sitúan las salidas visuales como posible evolución futura; el flujo actual genera PNG pero no los envía al modelo.
Alucinaciones y RAG	Fiabilidad	Reducción de respuestas no fieles mediante anclaje en fuentes o datos externos [16], [17].	El TFM adopta una lógica de anclaje en CSV locales, código ejecutado y salida JSON, aunque no implemente RAG documental.

Tabla 2.1: Comparativa entre trabajos relacionados y la propuesta del TFM.

En consecuencia, el posicionamiento del trabajo es deliberadamente intermedio. No se plantea un nuevo LLM financiero, ni un sistema autónomo de inversión, ni una plataforma multimodal completa. Se propone un prototipo trazable para análisis financiero histórico donde el modelo aporta interpretación, generación de código y redacción final, mientras el sistema conserva el control sobre validación, ejecución y artefactos. Esta combinación es la que da sentido al diseño descrito en los capítulos posteriores.

Capítulo 3

Datos y material de trabajo

3.1. Fuente de datos y datos obtenidos

El sistema trabaja con datos financieros históricos descargados previamente mediante `yfinance` [18] y almacenados en ficheros CSV. Yahoo Finance publica información de mercado, fichas de activos y series históricas para acciones, fondos cotizados, índices, divisas, criptomonedas y materias primas. La librería `yfinance` permite acceder de forma programática a parte de esta información disponible en Yahoo Finance [19], incluyendo campos habituales como apertura, máximo, mínimo, cierre, cierre ajustado y volumen. En este trabajo se utiliza como herramienta práctica de descarga para un contexto académico, no como proveedor garantizado para producción.

Cada CSV se conserva junto a un fichero de metadatos en formato JSON. Este metadato registra la consulta original, la intención analítica, los símbolos financieros implicados, el intervalo temporal y los parámetros de descarga. La decisión permite separar la fase de obtención de datos de la fase de análisis: el MVP no depende de que la API externa esté disponible durante cada ejecución, sino que opera sobre un conjunto de datos congelado y revisable.

La Tabla 3.1 muestra los datos obtenidos y utilizados como base del TFM. No se incluyen todos los nombres de fichero en la tabla para evitar una presentación demasiado técnica; los nombres completos se mantienen en `data/raw` y en el resumen procesado `data/processed/eda_resumen_ficheros.csv`.

Caso de datos	Activos	Cobertura	Uso en el MVP
NVDA cinco años	NVDA	2021–2026, diario	Crecimiento de precio y validación de respuestas de Nivel A.
NVDA frente a AMD	NVDA, AMD	2024–2026, diario	Comparación entre activos y cálculo de rentabilidad relativa.
AAPL tres meses	AAPL	2025–2026, diario	Visión descriptiva de un activo y análisis técnico básico.
QQQ frente a SPY	QQQ, SPY	2024, diario	Retorno, riesgo, drawdown y visualizaciones comparativas.
S&P 500 desde 2020	<code>^GSPC</code>	2020–2026, diario	Serie amplia de mercado para pruebas de histórico largo.
Bitcoin 2024	BTC-USD	2024, diario	Activo cripto con alta variabilidad.
EUR/USD horario	EURUSD=X	10 días, 1h	Ejemplo de divisa con frecuencia intradía.
Oro horario	GC=F	1 semana, 1h	Materia prima con frecuencia intradía.

Tabla 3.1: Datos obtenidos y finalidad de cada conjunto dentro del MVP.

La variedad de activos permite comprobar que el sistema no está limitado a una acción concreta. También fuerza al flujo LLM a trabajar con horizontes distintos, desde consultas diarias de varios años hasta series horarias de pocos días.

3.2. Catálogo de consultas y proceso de descarga

El punto de partida del conjunto de datos es el catálogo `data/catalog/query_cases.json`. Cada entrada define la consulta de referencia, la intención esperada, los tickers, el rango temporal o período relativo, el intervalo de observación y los posibles avisos. Este catálogo actúa como contrato entre la fase de preparación de datos y el flujo de análisis.

El proceso utilizado para construir el conjunto de datos puede resumirse en cuatro pasos:

1. Definir o revisar la consulta en el catálogo, por ejemplo una comparativa entre QQQ y SPY o una descarga de NVDA a cinco años.
2. Transformar esa consulta en parámetros de descarga: lista de tickers, `start`, `end`, `period` e `interval`.

3. Descargar las series con `yfinance` y guardar el resultado en `data/raw` como CSV, junto a un `.metadata.json` con los parámetros usados.
4. Ejecutar `scripts/generate_data_eda.py` para normalizar los CSV, generar tablas procesadas y producir el informe exploratorio auxiliar.

En el repositorio actual, los CSV crudos ya están congelados. Por tanto, para reproducir la parte documentada de esta memoria no es necesario volver a llamar a Yahoo Finance: basta con conservar `data/raw` y ejecutar el script de EDA si se quieren regenerar las tablas procesadas. Si se quisiera ampliar el conjunto de datos, el procedimiento correcto sería añadir primero la consulta al catálogo y después generar un nuevo CSV con su metadato asociado.

Cada consulta se transforma en una representación estructurada antes de entrar en el flujo LLM. El siguiente bloque muestra un ejemplo simplificado:

```
{
  "query": "Compárame QQQ y SPY en 2024 de forma clara,
           con tabla y gráfica normalizada",
  "tickers": ["QQQ", "SPY"],
  "csv_paths": [
    "data/raw/qqq_y_spy_desde_20240101_hasta_20241231.csv"
  ],
  "intent": "compare_assets",
  "start": "2024-01-01",
  "end": "2024-12-31",
  "period": null,
  "interval": "1d",
  "warnings": []
}
```

La Tabla 3.2 resume el significado de los campos principales.

Elemento	Descripción	Ejemplo
Consulta original	Texto introducido o definido como caso de prueba.	Comparar Nvidia y AMD.
Intención analítica	Tipo de análisis que debe ejecutarse.	Comparación de activos.
Símbolos financieros	Activos sobre los que se realiza el análisis.	NVDA, AMD.
Rango temporal	Período relativo o fechas de inicio y fin.	Dos años.
Intervalo	Frecuencia de los datos históricos.	Diario.
Avisos	Observaciones detectadas al preparar la entrada.	Sin avisos.

Tabla 3.2: Estructura semántica de una consulta financiera antes de su procesamiento.

3.3. Estructura de los CSV

Los ficheros generados por `yfinance` contienen columnas de tipo OHLCV: apertura, máximo, mínimo, cierre, cierre ajustado y volumen. Cuando hay más de un ticker, las columnas se guardan con cabeceras multinivel: el primer nivel identifica el símbolo financiero y el segundo nivel identifica el campo de mercado. Por este motivo, los CSV no se leen como tablas planas con una sola fila de cabecera, sino con `header=[0, 1]`.

La Tabla 3.3 resume los campos usados por el sistema. La columna de cierre es la variable principal, porque de ella dependen las métricas de crecimiento, rentabilidad, volatilidad, drawdown e indicadores técnicos básicos.

Campo	Uso principal	Observación
Open	Precio de apertura.	Se conserva como contexto, aunque no es obligatorio en todas las respuestas.
High y Low	Rango máximo y mínimo.	Útiles para revisar amplitud diaria o intradía.
Close	Cálculo financiero central.	Base de rentabilidad, volatilidad, drawdown y medias móviles.
Adj Close	Precio ajustado.	Se conserva si aparece en la descarga original.
Volume	Actividad negociada.	Puede ser cero o menos informativo en divisas y algunos futuros.

Tabla 3.3: Campos OHLCV presentes en los CSV de mercado.

La estructura multinivel también explica por qué el proyecto incluye una capa de

normalización. El script de EDA transforma cada combinación caso-ticker en formato largo, con columnas explícitas para `case_id`, `source_file`, `ticker`, `interval`, fecha y campos OHLCV. Este formato facilita auditar métricas por serie y evita que el código generado por el LLM tenga que resolver por sí solo todas las variantes de cabecera.

3.4. Análisis exploratorio de datos

El análisis exploratorio se realizó con `scripts/generate_data_eda.py`. Este script lee los CSV descargados con `yfinance`, interpreta las cabeceras multinivel, normaliza los datos a formato largo y genera tres salidas procesadas: `eda_dataset_largo.csv`, `eda_resumen_ficheros.csv` y `eda_resumen_series.csv`. Además, produce un informe textual en `docs/eda_datos_tfm.txt`.

El análisis confirma que el conjunto contiene 5.080 filas normalizadas, con un rango observado entre el 2 de enero de 2020 y el 17 de marzo de 2026. Los intervalos considerados son diario (1d) e intradía horario (1h).

Indicador	Resultado
Casos definidos en catálogo	10
CSV analizados	8
Series por símbolo y caso	10
Filas normalizadas	5.080
Nulos en precios de cierre	0
Duplicados por fecha	0

Tabla 3.4: Resumen de cobertura y calidad del conjunto de datos.

Caso	Tickers	Filas	Rango observado
AAPL tres meses	AAPL	60	2025-12-17 a 2026-03-16
NVDA frente a AMD	AMD, NVDA	500	2024-03-18 a 2026-03-16
NVDA cinco años	NVDA	1.255	2021-03-17 a 2026-03-16
Bitcoin 2024	BTC-USD	365	2024-01-01 a 2024-12-30
S&P 500 desde 2020	^GSPC	1.558	2020-01-02 a 2026-03-16
QQQ frente a SPY	QQQ, SPY	251	2024-01-02 a 2024-12-30

Tabla 3.5: Muestra de cobertura temporal obtenida en el EDA.

Además del resumen agregado, se calcularon métricas descriptivas por serie. La Tabla 3.6 muestra una selección de resultados. Se observan activos con crecimientos muy elevados, como NVDA en cinco años, series con caídas acumuladas significativas,

como AMD en la comparativa con NVDA, y activos de menor volatilidad relativa, como SPY. Esta diversidad obliga al sistema a redactar respuestas para escenarios con magnitudes y niveles de riesgo distintos.

Serie	Retorno total	Volatilidad anual	Máx. caída
NVDA, 5 años	1273.33 %	51.31 %	-66.36 %
BTC-USD, 2024	109.76 %	44.13 %	-26.18 %
NVDA, 2 años	107.13 %	49.42 %	-36.89 %
S&P 500, desde 2020	105.64 %	20.77 %	-33.93 %
QQQ, 2024	28.07 %	17.99 %	-13.56 %
SPY, 2024	24.45 %	12.63 %	-8.41 %
AMD, 2 años	3.11 %	55.10 %	-58.98 %
AAPL, 3 meses	-7.00 %	24.15 %	-10.07 %

Tabla 3.6: Métricas exploratorias por serie utilizadas para contextualizar los casos de prueba.

3.5. Calidad, cobertura y limitaciones

La ausencia de nulos en el precio de cierre es relevante porque las métricas principales del MVP dependen directamente de esta columna. También se comprobó la ausencia de duplicados por fecha en los ficheros analizados. Estas condiciones permiten ejecutar las intenciones actuales sin introducir imputaciones adicionales.

Como limitación, las descargas basadas en períodos relativos, por ejemplo tres meses o cinco años, dependen de la fecha en la que se generó el CSV. Para una versión final completamente cerrada sería recomendable fijar fechas explícitas de inicio y fin en todos los casos. También debe considerarse que los datos proceden de Yahoo Finance mediante `yfinance`. Al tratarse de una herramienta abierta no afiliada ni avalada por Yahoo, su uso se considera adecuado para un contexto académico y experimental, pero en un entorno productivo habría que estudiar licencias, disponibilidad, estabilidad del proveedor y derechos de uso de los datos descargados [18].

3.6. Material generado por el flujo y trazabilidad

Además de los CSV históricos, el proyecto genera material intermedio durante cada ejecución. Este material es importante porque permite revisar cómo una consulta en lenguaje natural se transforma en una respuesta final. En un TFM no basta con indicar que el sistema responde: también debe poder explicarse qué entendió el modelo, qué

cálculo se preparó, qué datos se usaron y qué resultado recibió el agente que redacta la explicación.

La Tabla 3.7 resume los artefactos conceptuales del flujo. La nomenclatura exacta puede evolucionar con la implementación, pero la separación de responsabilidades es estable: el primer agente no debe inventar cifras ni redactar la respuesta final, sino preparar un plan de análisis; la capa de cálculo produce resultados verificables; y el segundo agente convierte esos resultados en lenguaje natural.

Fase	Recibe	Genera	Utilidad en la memoria
Entrada normalizada	Consulta, tickers, rango temporal y rutas CSV.	FinancialQueryInput o estructura equivalente.	Fija qué datos puede usar el sistema y evita ambigüedades en la ejecución.
Agente 1	Entrada normalizada y catálogo de operaciones esperadas.	Plan de análisis con intención, métricas, salidas solicitadas y restricciones.	Permite evaluar si el modelo entendió la consulta antes de calcular nada.
Capa de cálculo	Plan validado y datos históricos preparados.	Métricas, tablas y datos de visualización derivados de los CSV.	Separa cálculo reproducible de redacción en lenguaje natural.
Agente 2	Consulta original, plan y resultados estructurados.	Respuesta final comprensible, prudente y sin recomendación de inversión.	Permite valorar claridad, fidelidad a los datos y reconocimiento de limitaciones.
Trazabilidad	Artefactos anteriores y estado de ejecución.	Registro revisable de entrada, plan, cálculo, errores y respuesta.	Facilita auditoría, depuración y comparación entre modelos.

Tabla 3.7: Material generado por el flujo y papel de cada agente.

Esta lectura también ayuda a distinguir material de trabajo de resultados finales. Los notebooks, informes Markdown y tablas procesadas se conservan como evidencias auxiliares para reconstruir decisiones de diseño, parámetros de descarga, estructura de los CSV, criterios de evaluación y salidas intermedias. No se presentan como conclusio-

nes por sí mismos, sino como soporte metodológico para que el flujo sea reproducible y revisable.

Los cuadernos auxiliares tienen funciones diferenciadas. El primero se utilizó como laboratorio para probar nuevos símbolos financieros, períodos e intervalos antes de incorporarlos al conjunto definitivo. El segundo documenta la relación entre consultas de referencia y CSV esperados. El tercero funciona como referencia principal del análisis exploratorio formal, apoyado por `scripts/generate_data_eda.py` y por las tablas agregadas usadas en esta memoria.

También se revisaron los documentos Markdown generados durante la evolución del proyecto. El documento de arquitectura sirvió para consolidar la descripción del flujo, la responsabilidad de cada agente y la decisión de separar comprensión, cálculo e interpretación. Los documentos de evaluación cualitativa y amplitud se emplearon para definir los niveles A/B/C; y las salidas de ejecución se reservaron como evidencias de apoyo para el capítulo de resultados y los anexos. Esta separación evita mezclar notas de desarrollo con resultados finales, pero mantiene la trazabilidad de las decisiones metodológicas.

Capítulo 4

Metodología

4.1. Enfoque de desarrollo

La metodología del proyecto combina desarrollo incremental de software, validación técnica del flujo y evaluación cualitativa de las respuestas generadas. No se plantea como un experimento de entrenamiento de modelos, sino como la construcción y revisión de un prototipo funcional que integra modelos LLM dentro de un sistema trazable de análisis financiero histórico. Por ello, la unidad principal de trabajo no es únicamente la respuesta final del modelo, sino el flujo completo que va desde la consulta del usuario hasta los artefactos generados y la explicación final.

El enfoque adoptado se basa en prototipado iterativo. En cada iteración se incorpora una capacidad concreta, se ejecuta sobre datos locales conocidos, se revisan los fallos y se ajustan los contratos entre componentes. Esta forma de trabajo encaja con la naturaleza del TFM, porque permite pasar de un caso mínimo demostrable a una arquitectura más completa sin perder control sobre las decisiones tomadas. Además, facilita justificar qué se valida en cada fase: entrada, datos, interpretación, código generado, ejecución, trazabilidad y respuesta final.

La metodología sigue una lógica incremental y reproducible:

1. Definir una entrada estructurada con consulta, tickers, fechas y ruta al CSV local.
2. Validar campos mínimos antes de invocar cualquier modelo LLM.
3. Interpretar la intención financiera de la consulta y transformarla en un plan analítico.
4. Generar código Python a partir del plan, del mensaje original y de los datos disponibles.
5. Validar el código antes de ejecutarlo, limitando importaciones, estructura y contrato de salida.

6. Ejecutar el script en un entorno controlado sobre CSV locales y guardar artefactos auditables.
7. Compactar salidas extensas cuando sea necesario para evitar saturar el contexto del segundo agente.
8. Usar el LLM interpretador para explicar únicamente métricas calculadas, sin añadir datos externos ni recomendaciones de inversión.
9. Revisar manualmente ejemplos representativos para valorar utilidad, trazabilidad y adecuación al usuario final.

Este enfoque separa tres planos. El plano de datos asegura que las consultas se resuelven sobre ficheros CSV conocidos. El plano de ejecución controla el código generado y sus salidas. El plano interpretativo convierte resultados estructurados en una explicación legible. La separación de estos planos reduce la opacidad habitual de una respuesta LLM directa y permite analizar con más precisión dónde se produce un acierto o un fallo.

4.2. Evolución incremental del MVP

El desarrollo del prototipo no se planteó como una implementación cerrada desde el inicio, sino como una sucesión de versiones del MVP. La primera versión tuvo como objetivo validar que el flujo completo podía funcionar de extremo a extremo. Para ello se trabajó con un cliente LLM compatible con una interfaz de tipo Chat Completions y con dos intencionalidades básicas: crecimiento de un activo y comparación entre activos. Esta fase permitió comprobar la viabilidad inicial de la cadena formada por consulta, interpretación, cálculo y respuesta.

Una vez validado ese flujo mínimo, el segundo MVP amplió la cobertura funcional de dos a seis intencionalidades analíticas: crecimiento de precio, comparación de activos, visión descriptiva de un activo, análisis de retornos, análisis de riesgo histórico y análisis técnico básico. Esta ampliación obligó a revisar contratos de entrada y salida, estructura de los planes analíticos, gestión de datos y formato de las respuestas.

Posteriormente, el proyecto evolucionó hacia la arquitectura actual, centrada en modelos LLM y generación de código controlada. En esta etapa se introdujeron validación de seguridad, ejecución aislada del script generado, guardado de artefactos, reintentos ante errores LLM y reparación del código cuando falla la validación o la ejecución. El sistema dejó de depender de funciones analíticas cerradas para cada caso y pasó a generar scripts específicos bajo contrato, lo que aumentó la flexibilidad del análisis pero también hizo necesaria una capa de control más estricta.

La última fase consistió en consolidar esa arquitectura y evaluarla con una batería progresiva de consultas A/B/C. Las consultas simples comprueban que el sistema responde a peticiones básicas; las consultas intermedias exigen tablas, comparaciones o visualizaciones; y las consultas avanzadas fuerzan informes más completos, varias métricas y salidas visuales. Esta evolución permite mostrar que el prototipo no sólo ejecuta ejemplos aislados, sino que puede aumentar la profundidad del análisis cuando el usuario lo solicita.

También se incorporó la comparación exploratoria entre varias configuraciones LLM:

- Groq con `llama-3.3-70b-versatile`.
- Gemini con `gemini-2.5-flash`.
- `openai/gpt-oss-20b` servido mediante vLLM.

Esta comparación no busca declarar un ganador estadístico, sino comprobar si la arquitectura mantiene el mismo contrato de entrada, generación, ejecución e interpretación al cambiar el modelo utilizado.

4.3. Criterios de validación

La validación del prototipo se organiza como una cadena de controles. Cada control intenta detectar un tipo de fallo antes de que afecte a la respuesta final. Esta decisión es importante porque, en un sistema con generación de código, no basta con revisar si el texto final parece razonable: también hay que comprobar que la entrada era válida, que los datos existían, que el código generado era seguro, que la ejecución produjo resultados estructurados y que la interpretación final se ajustó a esos resultados.

Nivel de validación	Qué comprueba	Riesgo que reduce
Entrada de usuario	Consulta no vacía, tickers, fechas, ruta al CSV y campos mínimos del esquema de entrada.	Evita lanzar el flujo con peticiones incompletas o imposibles de resolver.
Disponibilidad de datos	Existencia de los CSV locales, columnas esperadas y coherencia básica de la ventana temporal.	Evita interpretar como fallo del modelo un problema de datos o cobertura.
Plan analítico	Intención detectada, métricas solicitadas, activos implicados y nivel de profundidad esperado.	Reduce desalineamientos entre lo que pide el usuario y lo que calcula el sistema.
Código generado	Presencia de función <code>main()</code> , salida mediante JSON, importaciones permitidas y ausencia de operaciones no autorizadas.	Reduce riesgos de seguridad, ejecución arbitraria o salida no parseable.
Ejecución controlada	Resultado de ejecutar el script, errores capturados, métricas calculadas, tablas y rutas de artefactos.	Permite distinguir fallo de generación, fallo de ejecución y resultado válido.
Interpretación final	Uso exclusivo de resultados calculados, explicación clara, prudencia financiera y ausencia de recomendaciones de inversión.	Reduce alucinaciones, datos externos no justificados y conclusiones persuasivas no respaldadas.
Trazabilidad	Conservación de payloads, código generado, salidas JSON, gráficas y logs.	Permite auditar una respuesta y reconstruir el flujo completo.

Tabla 4.1: Criterios de validación aplicados al flujo del prototipo.

En la práctica, estos criterios permiten clasificar una ejecución con más detalle que una simple etiqueta de éxito o fracaso. Un caso puede fallar al inicio por credenciales o cuota del proveedor, puede llegar a generar un plan correcto pero producir código inválido, puede ejecutar el código pero devolver una salida incompleta, o puede completar la ejecución y aun así requerir revisión cualitativa de la explicación final. Esta granularidad es útil para el TFM porque convierte los fallos en información de diseño.

La validación no pretende garantizar que el sistema sea infalible ni sustituir una revisión experta. Su objetivo es acotar el comportamiento del prototipo y hacer que cada respuesta sea verificable. Por ello, las métricas y visualizaciones generadas se consideran evidencias intermedias, mientras que la respuesta final se interpreta como una capa de comunicación construida sobre esas evidencias. Esta separación es la base me-

metodológica que permite defender el uso de LLM en el proyecto sin tratar sus respuestas como una caja negra.

4.4. Criterios cualitativos de calidad del análisis

Además de la validación técnica, la metodología incorpora una revisión humana de la calidad del análisis generado. Esta revisión es necesaria porque dos ejecuciones pueden terminar con estado `completed` y, aun así, ofrecer niveles de utilidad distintos para el usuario. El objetivo no es obtener una puntuación matemática absoluta, sino valorar si la respuesta es completa, prudente, trazable y coherente con la consulta.

Se considera buen análisis aquel que responde directamente a la petición, calcula o muestra los datos esenciales, respeta el formato solicitado por el usuario y separa con claridad dato histórico, interpretación y limitaciones. Esta definición se apoya en recomendaciones de educación financiera que advierten que el rendimiento histórico debe presentarse indicando cómo se calcula y sin tratarlo como garantía de resultados futuros [20]. La selección de métricas parte de la relación entre riesgo y retorno [21] y de medidas habituales como rentabilidad, volatilidad y correlación en series de retornos históricos [22]. El drawdown se incluye como medida interpretable de caídas históricas porque permite observar pérdidas desde máximos previos usando datos observados [23].

Estas fuentes se usaron como guía para traducir una petición en lenguaje natural a una salida analítica razonable: primero se identifica si el usuario pide rentabilidad, comparación, riesgo, evolución temporal o explicación; después se seleccionan métricas coherentes con esa intención; y finalmente se exige que la interpretación distinga entre resultados históricos observados, lectura descriptiva y límites del análisis. De este modo, la elección de “buen análisis” no queda basada solo en preferencia subjetiva, sino en criterios documentados de presentación de rendimiento, relación riesgo-retorno y visualización comprensible.

La revisión diferencia dos responsabilidades. El primer agente no debe redactar una respuesta financiera final: su tarea es interpretar la consulta y generar código que produzca métricas, tablas o datos de visualización suficientes para responderla. El segundo agente no debe inventar datos ni completar huecos con conocimiento externo: su tarea es transformar la salida estructurada de la ejecución en una explicación comprensible, separando dato histórico, interpretación y limitaciones. De este modo, la calidad final depende de dos eslabones evaluables por separado: la calidad del material generado por el primer agente y la calidad de la explicación redactada por el segundo.

4.5. Niveles de profundidad del análisis

La profundidad del análisis se adapta a lo que solicita el usuario. Se definen tres niveles:

- **Nivel A, análisis simple por defecto:** se aplica cuando la consulta es sencilla y no pide formato especial. La salida esperada es un texto breve y una tabla o bloque estructurado con métricas básicas. No se fuerza la generación de gráficas si la respuesta puede entenderse con datos resumidos.
- **Nivel B, análisis mejorado:** se aplica cuando el usuario pide un análisis más completo, una comparativa clara o una explicación con más detalle. La salida esperada combina texto, tabla de métricas y una gráfica principal o los datos necesarios para representarla.
- **Nivel C, análisis profesional o enriquecido:** se aplica cuando el usuario pide expresamente un informe detallado, profesional, avanzado o con varias visualizaciones concretas. Puede incluir varias tablas, evolución normalizada, drawdown, retornos, volatilidad, correlación, volumen, medias móviles o desglose por períodos.

Esta graduación evita sobrecargar consultas simples y, al mismo tiempo, permite evaluar si el sistema es capaz de aumentar la riqueza del análisis cuando el usuario lo solicita. En series temporales financieras, los gráficos de línea son adecuados para mostrar evolución en el tiempo; en comparaciones entre activos con escalas distintas se prioriza la normalización, y se evita abusar de demasiadas series o ejes dobles para no inducir lecturas confusas [24].

En las consultas de Nivel B y C, el primer agente puede producir datos extensos para visualizaciones. Para evitar que esas series completas saturen el contexto del segundo agente, el workflow conserva la salida íntegra como artefacto auditable y envía al LLM interpretador una versión compactada. Esta decisión mantiene la trazabilidad técnica y, a la vez, reduce el riesgo de errores por exceso de contexto.

4.6. Evaluación con proveedores LLM

La evaluación se ha preparado para comparar tres configuraciones accesibles desde el mismo cliente LLM: Groq con `llama-3.3-70b-versatile` [25], modelo Llama 3.3 70B Instruct orientado a razonamiento general y baja latencia; Gemini con `gemini-2.5-flash` [26], modelo Flash de Gemini 2.5 orientado a rapidez y razonamiento; y `openai/gpt-oss-20b` [27] servido mediante un despliegue vLLM compatible con Chat Completions, modelo abierto de la familia gpt-oss usado por su capacidad

de razonamiento, seguimiento de instrucciones y generación de código. El proceso de depuración principal se documentó con `openai/gpt-oss-20b` [27]; posteriormente se comprobó la misma batería compleja con Groq usando `llama-3.3-70b-versatile` [25] y se realizaron pruebas puntuales con Gemini usando `gemini-2.5-flash` [26]. En todos los casos el flujo conceptual es el mismo: análisis de la consulta, generación de código, control de seguridad, ejecución e interpretación final. Las diferencias entre proveedores se observan en latencia, estabilidad, calidad textual y cumplimiento del formato esperado.

4.7. Revisión humana

El flujo no se plantea como una automatización ciega. La revisión humana posterior forma parte del proceso metodológico: permite comprobar si la interpretación inicial, el código generado, los resultados estructurados y la respuesta final son coherentes con la consulta del usuario y con los datos históricos utilizados.

La plantilla de revisión manual considera, como mínimo, comprensión de la consulta, selección de métricas, adecuación del código generado, cumplimiento de tablas o gráficas solicitadas, exactitud respecto a la salida ejecutada, claridad de la respuesta final, utilidad para el usuario, reconocimiento de limitaciones, ausencia de recomendaciones de inversión y trazabilidad entre consulta, código, métricas e interpretación. La valoración se expresa mediante categorías cualitativas, por ejemplo excelente, correcto, parcial o deficiente, para reconocer la subjetividad controlada de este tipo de evaluación.

Capítulo 5

Diseño del sistema

5.1. Arquitectura general

La arquitectura general se ha diseñado para evitar que el modelo de lenguaje actúe como una caja negra que recibe una pregunta financiera y devuelve directamente una respuesta textual. El flujo separa tres responsabilidades: comprender la consulta, calcular métricas financieras sobre datos históricos y explicar los resultados de forma comprensible. Esta separación es la base de la Figura 5.1.

El usuario no interactúa con un script ni con una tabla de datos, sino con una consulta en lenguaje natural. Por ello, el primer paso del sistema consiste en normalizar esa petición: identificar activos, período temporal, rutas de datos disponibles y tipo de análisis solicitado. A partir de esta entrada, el primer agente LLM interpreta la intención de la consulta y prepara el análisis que debe realizarse. Su salida no debe entenderse como una respuesta financiera final, sino como un plan de trabajo para la capa de cálculo.

Entre los dos agentes se sitúa una capa controlada y reproducible. Esta capa representa la parte determinista del sistema: valida que el plan sea posible, carga los precios históricos preparados, calcula métricas financieras conocidas y devuelve resultados estructurados. Su función es impedir que las cifras dependan de una improvisación textual del modelo. El LLM puede decidir qué necesita calcularse, pero las métricas se obtienen mediante procedimientos programáticos sobre los CSV del proyecto.

Finalmente, el segundo agente recibe resultados ya calculados y redacta la explicación final. Este agente no recalcula métricas, no añade datos externos y no debe completar huecos con conocimiento general del mercado. Su papel es comunicativo: transformar tablas, métricas y evidencias en una respuesta clara, prudente y adaptada al usuario.

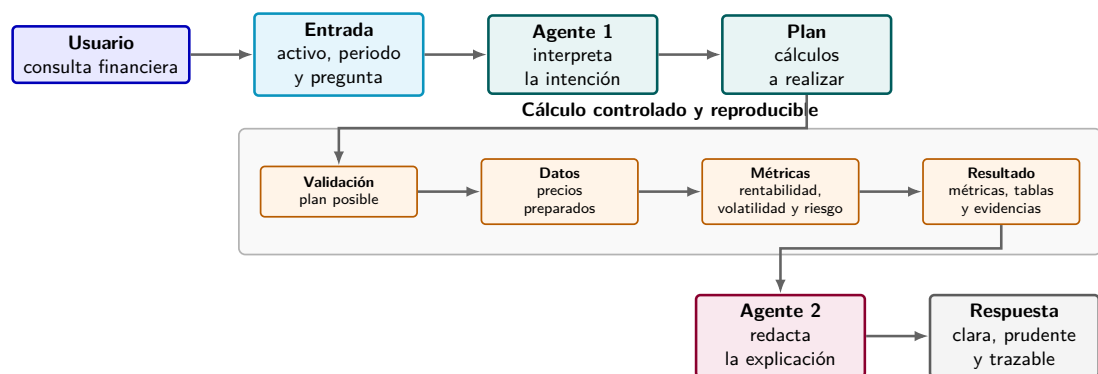


Figura 5.1: Arquitectura general del sistema desde la consulta del usuario hasta la respuesta trazable.

La lectura principal de la figura es que el LLM aparece en los extremos cognitivos del flujo: comprensión inicial y explicación final. El centro del sistema queda reservado para cálculo controlado. Esta decisión reduce variabilidad, mejora la trazabilidad y facilita comparar proveedores LLM, porque cada modelo se evalúa sobre las mismas entradas, los mismos datos históricos y un contrato de salida común.

Bloque	Responsabilidad	Salida esperada
Entrada normalizada	Convertir la consulta en campos revisables: activos, fechas, intervalo y CSV.	Estructura de entrada con los datos que el sistema puede usar.
Agente 1	Interpretar intención, nivel de profundidad y cálculos necesarios.	Plan de análisis o receta estructurada, sin cifras inventadas.
Capa de cálculo	Validar el plan, cargar datos históricos y ejecutar métricas conocidas.	Métricas, tablas y evidencias calculadas sobre los CSV.
Agente 2	Redactar una explicación fiel a los resultados calculados.	Respuesta final clara, prudente y sin recomendación de inversión.

Tabla 5.1: Responsabilidades principales de la arquitectura general.

La trazabilidad no se representa como un paso adicional del flujo principal, porque no es una transformación funcional de la consulta. Es una propiedad transversal de la ejecución. El sistema debe conservar la pregunta, el plan validado, los datos empleados, las métricas obtenidas, los errores si aparecen y la respuesta entregada. Esta información permite auditar una ejecución, repetir una prueba, comparar modelos y localizar en qué fase se produjo un fallo.

Desde el punto de vista metodológico, la arquitectura evita dos extremos. Por un lado, no se deja que el LLM improvise métricas financieras en texto libre. Por otro, tampoco se obliga al usuario a formular consultas como código o como operaciones manuales. El sistema utiliza el modelo para traducir lenguaje natural a una intención analítica, mantiene el cálculo bajo control programático y vuelve a usar lenguaje natural solo cuando ya existen resultados verificables.

En el prototipo actual, esa capa de cálculo se materializa mediante generación de código Python validado y ejecución controlada. Esta implementación permite comprobar la viabilidad del flujo extremo a extremo, explorar distintos modelos y estudiar errores reales de formato, seguridad y ejecución. Aunque en una evolución futura podrían fijarse determinadas operaciones analíticas como funciones cerradas, el MVP evaluado en esta memoria mantiene la generación de código como parte central del flujo: el primer agente prepara el análisis y el script, la capa intermedia valida y ejecuta, y el segundo agente explica.

La arquitectura se documentó también como artefacto interno de apoyo, recogido en el Anexo A.5. Ese material sirvió para fijar el mapa conceptual del flujo, la responsabilidad de cada nodo y la decisión de tratar cualquier salida generativa como una etapa intermedia validable, no como una respuesta final. En el cuerpo principal de la memoria se mantiene una única figura de arquitectura, orientada al lector final; el detalle técnico de nodos se desarrolla en las secciones siguientes para evitar duplicar esquemas y separar con claridad el flujo funcional de los artefactos de trazabilidad.

5.2. Nodos del flujo

El flujo implementado puede entenderse como una cadena de nodos con responsabilidades independientes. Algunos nodos dependen del modelo de lenguaje y otros son deterministas. Esta distinción es relevante porque permite cambiar de proveedor LLM sin alterar la validación de datos, el contrato del código ni la ejecución controlada.

- **Validación de entrada:** comprueba que la consulta sea procesable, que los tickers estén definidos, que las fechas sean coherentes y que existan los CSV históricos necesarios.
- **Agente 1: planificación:** interpreta la intención del usuario, identifica el nivel de análisis A/B/C y define qué métricas, tablas o datos visuales deben obtenerse.
- **Agente 1: generación de código:** transforma el plan analítico en un script Python autocontenido, ajustado al contrato de entrada y salida del sistema.
- **Validación y reparación:** revisa importaciones, estructura mínima, uso de `main()`, lectura de `argv[1]` y emisión de JSON. Si el código no cumple el contrato

o falla de forma corregible, puede solicitarse una reparación al modelo.

- **Ejecución controlada:** ejecuta el script en un proceso separado sobre datos locales, capturando salida estándar, errores, métricas, tablas y posibles artefactos.
- **Compactación de resultados:** reduce salidas largas antes de enviarlas al segundo agente, por ejemplo series temporales extensas o datos de visualización, manteniendo la salida completa como evidencia auditable.
- **Agente 2: interpretación:** recibe únicamente resultados ya calculados y redacta una explicación en español, con limitaciones y sin recomendaciones de inversión.

La separación entre nodos no es sólo organizativa. También permite evaluar mejor el comportamiento del sistema. Un caso puede fallar porque el proveedor LLM no responde, porque el plan es insuficiente, porque el código generado no cumple el contrato, porque la ejecución produce un error o porque la interpretación final no se completa. Al conservar esa información por fases, la evaluación no se limita a contar respuestas correctas o incorrectas, sino que identifica en qué parte del flujo se produce cada incidencia.

5.3. Contrato del código generado

El contrato del código generado es la interfaz entre la parte generativa del sistema y la parte determinista. Su función es convertir una salida libre del LLM en un artefacto que pueda revisarse, ejecutarse y comparar de forma sistemática. Sin este contrato, cada script podría tener una estructura distinta, imprimir texto no parseable, depender de recursos externos o mezclar cálculo con explicación, lo que haría difícil auditar el resultado.

El código generado no debe redactar la respuesta final. Su responsabilidad es calcular métricas y devolver una salida estructurada. El contrato mínimo exige una función `main()`, lectura de los datos de entrada recibidos por `argv[1]`, escritura de un único JSON por `stdout` y uso exclusivo de importaciones permitidas. Además, el script debe trabajar con los CSV locales proporcionados por el sistema, no descargar información externa ni depender de estado global. Estas reglas permiten que todos los scripts se traten de la misma forma: primero se validan, después se ejecutan en un proceso separado y finalmente se interpreta su salida JSON.

La salida JSON puede incluir métricas escalares, tablas, series preparadas para gráficas, avisos y rutas de artefactos generados. Esta estructura permite que una consulta simple produzca una respuesta breve y que una consulta de nivel B o C genere evidencias más ricas, como comparativas, evolución normalizada o drawdown. Por tanto, el primer agente no se evalúa por lo convincente que suene una explicación, sino por si

prepara cálculo suficiente para la consulta. Del mismo modo, el segundo agente no se evalúa por calcular nuevas métricas, sino por interpretar fielmente lo que el script ha devuelto.

El sentido práctico de este contrato es reducir el riesgo de tres problemas habituales en sistemas LLM con generación de código: ejecución de instrucciones no deseadas, respuestas imposibles de procesar automáticamente y pérdida de trazabilidad entre consulta, código, métricas y explicación final. Por ello, el contrato no es solo una restricción técnica, sino una decisión metodológica: obliga a que el modelo produzca cálculo verificable antes de producir lenguaje natural.

5.4. Gestión de errores

Los errores se separan por origen: entrada inválida, error de proveedor LLM, salida con formato incorrecto, código rechazado, error de ejecución y error de interpretación. Esta separación facilita la depuración y la revisión humana posterior, porque permite saber si el problema procede de los datos, del proveedor, del contrato de código, del entorno de ejecución o de la respuesta final.

En la fase inicial, una consulta puede rechazarse si faltan tickers, fechas o ficheros de datos. En las fases LLM, el sistema registra errores de conectividad, cuota, credenciales, respuestas vacías o JSON no parseable. En la fase de generación de código, el script puede ser rechazado antes de ejecutarse si utiliza importaciones no permitidas, no define `main()`, no lee la entrada esperada o no emite un JSON único. Si el script supera esa validación, todavía puede fallar durante la ejecución por errores de cálculo, columnas ausentes o serialización incorrecta.

El sistema no oculta estos fallos bajo una respuesta aparentemente válida. Cuando existe una ejecución parcial, se conserva el estado alcanzado, el error observado y las evidencias disponibles. Esta decisión es importante para la evaluación: un límite de cuota de un proveedor no tiene el mismo significado que un error de razonamiento, y un fallo de formato no debe confundirse con una métrica financiera incorrecta. La memoria, por tanto, analiza la completitud del flujo junto con el punto exacto en el que se detiene cada caso.

Capítulo 6

Implementación

6.1. Estructura del código

La implementación se organiza en módulos pequeños:

Módulo	Responsabilidad
<code>schemas.py</code>	Define la entrada, el plan interpretado, los artefactos y el estado del workflow.
<code>validation.py</code>	Valida query, tickers, fechas y rutas CSV antes de llamar al LLM.
<code>nodes.py</code>	Implementa los nodos del flujo: análisis LLM, codegen, seguridad, ejecución e interpretación.
<code>src/llm/</code>	Contiene prompts, cliente LLM compatible con Chat Completions y funciones del pipeline LLM.
<code>code_security.py</code>	Revisa el código generado antes de ejecutarlo.
<code>code_runner.py</code>	Guarda el script, lanza la ejecución y captura stdout, stderr y artefactos.
<code>market_data.py</code>	Proporciona funciones auxiliares para leer CSV financieros descargados con <code>yfinance</code> .
<code>tests/</code>	Comprueba el workflow con llamadas LLM simuladas y escenarios de error.

Tabla 6.1: Módulos principales de la implementación.

6.2. Repositorio y organización del trabajo

El desarrollo se ha gestionado en un repositorio GitHub disponible en <https://github.com/CarlosRuiz4102/tfm>. El uso de control de versiones permite conservar la evolución del código, la memoria, los scripts de evaluación y la documentación auxiliar en un único espacio de trabajo. Esta organización facilita revisar qué parte corresponde a datos, qué parte corresponde al flujo LLM, qué parte corresponde a resultados y qué parte corresponde a documentación académica.

La estructura del repositorio separa responsabilidades de forma cercana a una práctica MLOps ligera:

- **data/**: contiene el catálogo de casos, los CSV históricos congelados y sus metadatos.
- **src/**: agrupa el código de aplicación, dividido en esquemas, grafo, integración LLM y ejecución controlada.
- **scripts/**: incluye utilidades para generar EDA, ejecutar evaluaciones y producir informes reproducibles.
- **tests/**: valida el workflow con respuestas LLM simuladas, evitando depender de APIs externas durante las pruebas básicas.
- **docs/**: conserva la memoria LaTeX, informes Markdown, figuras y registros de resultados.
- **results/**: almacena artefactos generados durante ejecuciones, como scripts, datos de entrada, registros y salidas JSON.

También se separan configuración y secretos. El fichero `.env` contiene claves locales y queda excluido del control de versiones, mientras que `.env.example` documenta las variables necesarias para reproducir la configuración sin exponer credenciales. Del mismo modo, `.gitignore` excluye entornos virtuales, cachés, registros y resultados generados de gran tamaño, manteniendo versionados los artefactos que aportan trazabilidad académica.

Aunque el proyecto no implementa un ciclo MLOps productivo completo con despliegue continuo, sí incorpora prácticas propias de ese enfoque: versionado del código, separación entre datos, código y resultados, scripts de evaluación repetibles, pruebas automatizadas, conservación de artefactos, documentación de configuraciones y registro explícito de fallos. Para el alcance del TFM, estas prácticas son suficientes para defender que el sistema no se limita a una demo puntual, sino que se ha organizado como un prototipo reproducible y auditable.

6.3. Contratos de datos

La entrada del sistema se representa mediante `FinancialQueryInput`. La interpretación generada por el LLM se almacena en `AnalysisPlan`, que contiene intencionalidad interpretada, tipo de análisis, métricas esperadas, requisitos de datos, requisitos de salida, preferencias de presentación y razonamiento resumido. El estado completo se conserva en `WorkflowState`.

La necesidad de funciones auxiliares de datos apareció durante las pruebas de desarrollo resumidas en `docs/planificacion_evaluacion_tfm.md`: los CSV generados por `yfinance` pueden contener cabeceras multinivel y formatos distintos según el activo o el intervalo. Para reducir errores repetidos en el código generado, se incorporaron funciones reutilizables de carga y normalización. Así, el LLM no tiene que resolver desde cero cada variante de lectura de CSV y puede concentrarse en el cálculo solicitado.

6.4. Pipeline LLM

El pipeline LLM se divide en tres llamadas principales. La primera interpreta la consulta y construye un plan estructurado. La segunda genera código Python usando tanto el plan como el mensaje original del usuario. La tercera interpreta la salida JSON producida por el script y redacta la respuesta final.

El prompt de generación de código exige que el script lea un payload por `argv[1]`, use librerías permitidas y escriba un único JSON en `stdout`. Esta restricción permite tratar el código generado como un artefacto ejecutable, auditable y validable.

El flujo completo puede resumirse como una cadena de transformación controlada:

prompt → código Python → validación de seguridad → ejecución →
JSON → interpretación final.

Esta cadena es importante porque el LLM no produce directamente una afirmación financiera final sin pasar antes por cálculos ejecutados. El primer bloque LLM prepara el análisis y genera el script; el entorno de ejecución produce datos estructurados; y el segundo bloque LLM redacta una explicación basada en esa salida.

Elemento	Contrato exigido
Entrada	El script recibe un único payload JSON por <code>argv[1]</code> , con consulta, tickers, fechas, intervalo y rutas CSV.
Lectura de datos	El código debe leer únicamente los CSV proporcionados por el workflow y no descargar datos externos.
Salida	Debe escribir un único objeto JSON en <code>stdout</code> , mediante <code>json.dumps</code> , para que el sistema pueda analizarlo.
Contenido	La salida debe incluir métricas, tablas o datos de visualización necesarios para responder a la consulta.
Artefactos	La ejecución conserva script, salida estándar, salida de error, código de retorno y metadatos de ejecución.
Errores	Si el script falla, el error queda registrado y el workflow puede devolver una explicación controlada o intentar reparación.

Tabla 6.2: Contrato operativo del código Python generado por el LLM.

6.5. Control de seguridad

Antes de ejecutar el script, el módulo `code_security.py` analiza el código con el AST de Python. El validador comprueba importaciones permitidas, llamadas bloqueadas como `eval` o `exec`, presencia de `main()` y uso de `json.dumps`. Si el código no cumple el contrato, el workflow termina con estado de error y no ejecuta el script.

El uso del árbol de sintaxis abstracta permite revisar la estructura del programa antes de ejecutarlo, en lugar de depender solo de búsquedas textuales. La validación comprueba nodos de importación, llamadas a funciones y elementos mínimos del contrato. Con ello se reduce el riesgo de que un script generado por LLM ejecute operaciones fuera del alcance del MVP. Esta capa no convierte el entorno en una sandbox completa, pero sí añade una barrera explícita entre generación y ejecución.

Comprobación	Finalidad
Imports permitidos	Limitar el código a librerías necesarias para análisis de datos y serialización.
Funciones bloqueadas	Evitar llamadas peligrosas o no necesarias, como <code>eval</code> y <code>exec</code> .
Función <code>main()</code>	Forzar una estructura ejecutable y sencilla de auditar.
Salida JSON	Garantizar que el resultado pueda pasar al nodo de interpretación sin ambigüedad.
Errores explícitos	Separar fallos de seguridad, fallos de ejecución y fallos de interpretación.

Tabla 6.3: Comprobaciones principales del validador de seguridad.

6.6. Reparación y compactación

Cuando el código generado no supera la validación o falla durante la ejecución, el workflow puede solicitar al LLM una reparación manteniendo la consulta original, el plan analítico y el detalle del error. La reparación no evita la validación: el código corregido vuelve a pasar por el mismo control de seguridad antes de ejecutarse. Si la reparación tampoco cumple el contrato, el caso se marca como error controlado.

En consultas de Nivel B o C, el script puede generar series visuales largas. Estas series son útiles como artefacto completo, pero pueden saturar el contexto del segundo agente. Por ello, antes de la interpretación final se compactan listas y textos extensos: se conserva una muestra representativa para el LLM interpretador y la salida completa queda guardada como evidencia auditable. Esta decisión permite mantener trazabilidad sin degradar la respuesta final por exceso de contexto.

Este ajuste quedó documentado en una nota interna de control de tamaño de sa-

lidas, referenciada en el Anexo A.5. La decisión práctica fue limitar o resumir datos de visualización enviados al segundo agente, manteniendo siempre la salida completa en los registros. De este modo, las consultas de Nivel B y C pueden producir material gráfico sin convertir la interpretación final en una llamada demasiado grande para el proveedor LLM.

6.7. Pruebas

Las pruebas automatizadas simulan las respuestas LLM para evitar dependencias externas durante su ejecución. De este modo se validan el workflow, el control de seguridad, la ejecución controlada y los errores de entrada sin consumir APIs reales.

Capítulo 7

Integración de modelos de lenguaje

7.1. Papel del LLM en el sistema

El trabajo incorpora modelos de lenguaje como componentes orquestados dentro de un flujo de análisis financiero histórico. El LLM no se utiliza como fuente directa de cifras ni como asesor financiero, sino como motor de comprensión, planificación, generación de código e interpretación textual. Las métricas se obtienen mediante ejecución programática sobre datos locales, y la respuesta final se redacta a partir de esos resultados ya calculados.

Esta organización permite aprovechar la flexibilidad del modelo sin ejecutar sus salidas de forma ciega. Entre la generación y la ejecución existe una capa de control que revisa el script, limita las operaciones permitidas y exige una salida JSON estructurada. De este modo, el LLM aporta adaptación al lenguaje natural, pero el sistema conserva trazabilidad y control sobre las operaciones financieras.

7.2. Separación entre agentes y prompts

Aunque el sistema usa un mismo cliente LLM configurable, en la arquitectura se distinguen dos agentes funcionales. El primer agente actúa como analista operativo: interpreta la consulta, decide qué métricas hacen falta y genera el código Python que implementa el cálculo. El segundo agente actúa como intérprete para usuario final: recibe resultados ya calculados y los explica en lenguaje claro. Esta separación evita que razonamiento, cálculo y redacción queden mezclados en una única respuesta no verificable.

La diferencia entre ambos agentes no está sólo en el orden de ejecución, sino en el contrato del prompt. El primer agente debe devolver estructuras formales, primero un `AnalysisPlan` y después un JSON con el campo `code`. No debe redactar la respuesta final. El segundo agente recibe el `execution_output` y el plan, pero no recibe permiso

para inventar datos, recalcular con fuentes externas ni recomendar inversión.

Bloque	Entrada principal	Salida exigida por prompt
Agente 1: planificación	Consulta validada, tickers, fechas, CSV disponibles y reglas A/B/C.	JSON con intención interpretada, tipo de análisis, métricas, columnas, requisitos de datos, requisitos de salida y razonamiento.
Agente 1: generación de código	Consulta original, entrada normalizada y plan analítico.	JSON con un campo <code>code</code> ; el script debe leer <code>argv[1]</code> , usar cargadores autorizados y escribir un único JSON en <code>stdout</code> .
Reparación asistida	Código previo, plan y error observado.	Nuevo JSON con <code>code</code> , manteniendo objetivo, nivel A/B/C y restricciones de seguridad.
Agente 2: interpretación	Salida ejecutada y plan analítico.	Respuesta breve en español, fiel a las métricas recibidas, con limitaciones y sin recomendación de inversión.

Tabla 7.1: Diferencias entre los prompts usados en cada bloque LLM.

7.3. Pipeline del primer y segundo agente

El pipeline del primer agente transforma una petición de usuario en un artefacto ejecutable. Primero recibe la entrada normalizada por el sistema, donde ya se han fijado tickers, fechas, intervalo y rutas CSV. Después produce un plan analítico en JSON. Con ese plan y el mensaje original, una segunda llamada LLM genera el script Python. El código pasa por validación de contrato y seguridad antes de ejecutarse. Si falla la validación o la ejecución, el sistema puede invocar el prompt de reparación, pero el código corregido vuelve a pasar por el mismo control y no se ejecuta automáticamente sin revisión programática.

entrada validada → prompt de análisis → AnalysisPlan → prompt
de codegen → script Python → validación → ejecución JSON

El pipeline del segundo agente empieza cuando ya existe una salida estructurada. Su entrada no es el CSV ni el código, sino el resultado ejecutado y una versión compactada cuando existen series largas. A partir de ahí redacta la respuesta final, explicando qué significan las métricas, qué limitaciones tiene el análisis y cómo leer las tablas o datos visuales.

salida JSON ejecutada → compactación si procede → prompt de
interpretación → respuesta final

Los extractos completos de prompts y tres ejecuciones representativas A/B/C se incluyen en el Anexo, para que pueda verse qué entra y qué sale de cada bloque sin sobrecargar el cuerpo principal de la memoria.

7.4. Cliente compatible con Chat Completions

La integración se ha preparado mediante un cliente que sigue una interfaz de tipo Chat Completions. Esta decisión permite alternar entre proveedores sin modificar el workflow principal. En la memoria, esta compatibilidad se refiere al formato de llamada, a los mensajes y a la forma de recibir respuestas; no significa que todos los modelos utilizados pertenezcan a OpenAI. El sistema utiliza variables de entorno para seleccionar perfil, modelo, URL base y clave de API.

En esta memoria, cuando se comentan resultados ya ejecutados, se hace referencia al modelo `openai/gpt-oss-20b` [27] servido mediante un despliegue vLLM compatible con Chat Completions. Este modelo abierto de la familia gpt-oss se usa por su capacidad de razonamiento, seguimiento de instrucciones y generación de código Python. El prefijo `openai/` forma parte del identificador del modelo, no implica que las demás configuraciones sean modelos de OpenAI.

La configuración operativa de los proveedores se mantiene separada de la lógica del sistema mediante variables de entorno. Cada perfil define proveedor, URL base, modelo y credenciales, lo que evita mezclar resultados del workflow con detalles locales de despliegue. Esta separación fue especialmente útil al repetir evaluaciones con Groq, Gemini y `openai/gpt-oss-20b` servido mediante vLLM sin cambiar la arquitectura del sistema.

7.5. Configuraciones evaluadas

Se han preparado tres configuraciones principales:

- **Groq con llama-3.3-70b-versatile** [25]: proveedor externo accesible desde el cliente común. El modelo corresponde a Llama 3.3 70B Instruct y se utiliza por su razonamiento general, seguimiento de instrucciones y baja latencia en las pruebas.
- **Gemini con gemini-2.5-flash** [26]: proveedor secundario para estudiar portabilidad y calidad comparativa. El modelo Flash de Gemini 2.5 se utiliza por su rapidez, capacidad de razonamiento y equilibrio entre latencia y calidad.
- **openai/gpt-oss-20b** [27]: modelo abierto de la familia gpt-oss servido mediante vLLM, compatible con Chat Completions. Se usa como modelo principal por su

capacidad de razonamiento, seguimiento de instrucciones y generación de código.

7.6. Gestión de errores de API

El diseño registra errores de red, cuota, indisponibilidad, credenciales o formato de respuesta. Si una llamada LLM falla, el sistema devuelve un error explícito para que la evaluación no mezcle resultados reales de modelo con respuestas programáticas.

La gestión de errores se concentra en el cliente LLM común y en la capa de pipeline. El cliente encapsula las excepciones de la API en una excepción propia, `LLMClientError`, de modo que el resto del workflow no depende de detalles internos de cada proveedor. Esta abstracción permite tratar de forma homogénea fallos de conectividad, claves ausentes, límites de cuota, tiempos de espera agotados o respuestas vacías, aunque el origen técnico sea diferente en Groq, Gemini o el despliegue que sirve `openai/gpt-oss-20b` mediante vLLM.

Además, no todos los errores LLM tienen el mismo significado dentro del sistema. En la fase de análisis, un fallo impide construir el plan inicial y el caso termina como error controlado. En la fase de generación de código, el error puede deberse a una respuesta no válida, a JSON mal formado o a un script que no cumple el contrato mínimo. En la fase de interpretación final, en cambio, puede existir una ejecución correcta y métricas calculadas, pero fallar la redacción final del modelo. Por este motivo el estado del workflow distingue entre peticiones inválidas, código rechazado, fallo de ejecución y casos `completed_with_error`.

Para reducir fallos por formato, el pipeline solicita JSON estricto en las fases de análisis y generación de código. Si la respuesta no puede parsearse, se realizan hasta tres intentos con un mensaje de corrección que pide devolver exclusivamente un objeto JSON válido. En la generación de código, además, se comprueba que el script contenga una función `main()` y que escriba un único JSON mediante `json.dumps`. Si el código generado no supera la validación de seguridad o falla durante la ejecución, el sistema puede pedir una reparación al LLM usando el error observado como contexto. Esta reparación queda registrada mediante avisos para no ocultar que el caso necesitó una segunda intervención del modelo.

Los prompts, el contrato de salida y el tratamiento de errores se revisaron a partir de ejecuciones reales del prototipo. En esas pruebas se observó que algunos fallos no se debían a una mala comprensión financiera, sino a detalles operativos: JSON incompleto, código no ejecutable, lectura incorrecta de CSV o salidas demasiado extensas. Por ello se combinaron reintentos de formato, reparación de código y compactación de salidas antes de la interpretación final.

En las evaluaciones reales se observaron varios tipos de fallo útiles para el análisis del TFM:

- **Errores de cuota:** por ejemplo, respuestas 429 cuando un proveedor supera el límite diario de uso. Estos casos no indican que el razonamiento del modelo sea incorrecto, sino que la comparación experimental queda condicionada por disponibilidad.
- **Errores de conexión:** pueden aparecer si la ejecución se lanza sin acceso de red real o si el proveedor no responde. En la documentación de resultados se marcan como diagnóstico de entorno y no como evidencia de rendimiento del modelo.
- **Errores de formato:** se producen cuando el modelo entiende la tarea, pero devuelve texto o código fuera del JSON esperado. Estos fallos justifican los reintentos y la validación posterior.
- **Errores por contexto o tamaño de salida:** algunas consultas visuales generan estructuras largas, por ejemplo series temporales completas. Para mitigarlo, antes de la interpretación final se compactan listas y textos extensos, manteniendo la salida completa como artefacto auditable.

Esta gestión de errores es importante metodológicamente. La memoria no presenta únicamente tasas de acierto, sino también las condiciones bajo las que falla el sistema. Separar errores de proveedor, errores de formato, errores de seguridad y errores de ejecución permite interpretar mejor los resultados y evita atribuir al modelo un problema que en realidad pertenece a la cuota, a la conectividad o al contrato técnico de salida.

7.7. Gestión de credenciales

Las claves o tokens de Groq con `llama-3.3-70b-versatile` [25], Gemini con `gemini-2.5-flash` [26] y `openai/gpt-oss-20b` [27] servido mediante vLLM se gestionan mediante variables de entorno en el fichero local `.env`. Este archivo no se versiona y no debe compartirse. El repositorio incluye configuraciones de ejemplo, pero una ejecución real con API solo es posible cuando el entorno local contiene una clave válida.

Capítulo 8

Resultados y evaluación

Este capítulo presenta la evaluación exploratoria realizada sobre el prototipo. El objetivo no es demostrar que el sistema prediga el mercado ni establecer un ranking definitivo de modelos, sino comprobar si la arquitectura interpreta consultas financieras históricas, genera código Python adecuado, valida su seguridad, ejecuta métricas trazables y redacta conclusiones comprensibles.

La evaluación combina una batería progresiva de consultas A/B/C, revisión cualitativa de las respuestas y análisis de artefactos generados por el workflow. Los resultados deben leerse como evidencia observada bajo condiciones concretas de datos, prompts, cuota y disponibilidad de proveedores.

8.1. Objetivo de la evaluación

El objetivo principal es verificar el comportamiento extremo a extremo del flujo:

- interpretación de la consulta y construcción de un plan;
- generación de código Python a partir del plan y del mensaje original;
- validación de seguridad antes de ejecutar;
- ejecución controlada sobre CSV locales;
- generación de métricas, tablas o datos de visualización;
- interpretación final sin predicción ni recomendación de inversión.

Se evaluaron tres configuraciones desde el mismo cliente LLM:

- Groq con `llama-3.3-70b-versatile` [25].
- Gemini con `gemini-2.5-flash` [26].

- `openai/gpt-oss-20b` [27], servido mediante vLLM en un despliegue compatible con Chat Completions.

En todos los casos el workflow usa el modelo configurado para análisis de consulta, generación de código e interpretación final. Si falta una clave, el proveedor no está disponible, el código se rechaza o la ejecución falla, el caso queda registrado como error controlado.

8.2. Métricas y criterios

Para comparar modelos no basta con comprobar si una ejecución termina. En este proyecto una configuración LLM puede completar el flujo y, aun así, producir un análisis pobre: por ejemplo, puede calcular sólo una rentabilidad acumulada cuando la consulta pedía riesgo, comparación y visualización. También puede ocurrir lo contrario: una ejecución puede fallar en la interpretación final, pero haber generado un plan y un código útiles. Por ello, la evaluación se divide en dos niveles complementarios: métricas técnicas del workflow y criterios cualitativos del análisis.

Las métricas técnicas registran la robustez operativa del sistema:

- **Casos completados:** consultas válidas que terminan con estado `completed`.
- **Errores controlados:** consultas inválidas o fallidas que terminan con explicación.
- **Errores LLM:** problemas de configuración, red, cuota, disponibilidad, contexto o formato.
- **Código rechazado:** scripts que no superan el contrato de seguridad.
- **Errores de ejecución:** scripts válidos que fallan al calcular o serializar la salida.
- **Violaciones de recomendación de inversión:** presencia de recomendaciones o predicciones fuera del alcance del MVP.
- **Tiempo de ejecución:** duración observada por caso y proveedor.

Estas métricas permiten saber si el modelo fue capaz de atravesar el pipeline, pero no determinan por sí solas qué LLM realiza mejores análisis. Para valorar la calidad analítica se revisan dimensiones adicionales, aplicadas sobre la misma consulta, los mismos datos y el mismo contrato de salida.

Dimensión evaluada	Criterio de valoración
Complejidad del flujo	Comprueba si el caso llega a planificación, generación de código, ejecución e interpretación final. Mide robustez operativa, no calidad analítica por sí sola.
Comprensión de la consulta	Revisa si el agente identifica activos, periodo, nivel A/B/C, restricciones y formato solicitado. Permite detectar respuestas genéricas o incompletas.
Selección de métricas	Valora si las métricas calculadas son pertinentes para la consulta: rentabilidad, volatilidad, drawdown, correlación, ranking mensual, evolución normalizada u otras.
Calidad del código y datos generados	Evalúa si el script usa correctamente los CSV, calcula lo necesario, produce JSON estructurado y prepara tablas o series visuales cuando procede.
Fidelidad de la interpretación	Comprueba si el segundo agente explica sólo los resultados ejecutados, sin inventar datos externos ni modificar cifras.
Claridad y utilidad para el usuario	Revisa si la respuesta está ordenada, es comprensible, reconoce limitaciones y ayuda a interpretar comportamiento histórico y riesgo.
Prudencia financiera	Verifica si evita predicciones, recomendaciones de compra/venta y lenguaje de asesoramiento, manteniendo las restricciones éticas del MVP.

Tabla 8.1: Dimensiones usadas para valorar la calidad técnica y analítica de cada ejecución.

La comparación entre modelos se interpreta, por tanto, como una comparación de comportamientos observados. Un modelo puede ser mejor en completitud, otro en claridad textual y otro en generación de tablas o datos visuales. El resultado relevante para la memoria no es declarar un ganador absoluto, sino identificar qué configuración produce análisis más completos y útiles bajo las mismas condiciones, y qué anomalías aparecen en la batería de ejemplos.

Esta decisión es coherente con la necesidad de explicar cómo se calcula y presenta el rendimiento histórico [20], y con la separación entre retorno, riesgo, volatilidad y correlación como conceptos de análisis histórico [21], [22]. La revisión cualitativa separa el comportamiento del primer agente, que debe elegir métricas adecuadas y generar datos suficientes, del segundo agente, que debe redactar una interpretación clara, prudente y fiel a la salida ejecutada.

8.3. Niveles cualitativos de salida

La evaluación se estructura en tres niveles:

- **Nivel A:** consultas simples sin requisitos de presentación. La respuesta esperada

es breve y debe incluir métricas básicas suficientes.

- **Nivel B:** consultas que piden un análisis más completo. La respuesta esperada incluye texto, tabla de métricas y una gráfica principal o datos estructurados para generarla.
- **Nivel C:** consultas profesionales o detalladas. La respuesta esperada incluye varias salidas estructuradas, por ejemplo tabla, evolución normalizada, drawdown, clasificación mensual, desglose de riesgo o conclusión ejecutiva.

Para series temporales se priorizan gráficos de línea y, cuando se comparan activos con escalas distintas, una evolución normalizada permite comparar la tendencia sin confundir niveles de precio absolutos. También se evita abusar de demasiadas series o ejes dobles, porque pueden dificultar la lectura [24]. En consultas de riesgo se considera especialmente útil el drawdown, ya que resume caídas históricas desde máximos previos y se entiende con facilidad [23].

Nivel	Uso previsto	Salida esperada
A	Query simple sin formato específico	Texto breve y métricas básicas en tabla o bloque estructurado.
B	Análisis más completo o comparativa clara	Texto, tabla de métricas y una gráfica principal o datos para representarla.
C	Petición profesional, detallada o con varias salidas	Texto, tablas, varias visualizaciones o datos estructurados, conclusión y limitaciones.

Tabla 8.2: Niveles cualitativos de profundidad del análisis.

8.4. Batería progresiva

La batería principal contiene 15 consultas: cinco de Nivel A, cinco de Nivel B y cinco de Nivel C. Incluye crecimiento individual, comparación entre activos, visión descriptiva, retornos, riesgo histórico, criptoactivos, divisas intradía, materias primas e informes con varias salidas. La definición completa se conserva en `docs/bateria_evaluacion_15_queries.md`.

Dentro del Nivel C, la quinta consulta es una query de estrés visual que obliga al sistema a respetar cuatro bloques de salida:

Compara QQQ y SPY durante 2024 como si fuera un informe para un cliente no técnico. Quiero que me muestres los datos exactamente en cuatro bloques: 1) una tabla resumen con rentabilidad total, volatilidad anualizada, máximo drawdown, mejor mes y peor mes; 2) una tabla de clasificación

mensual indicando qué activo ganó cada mes; 3) datos para una gráfica normalizada base 100 de ambos activos; 4) datos para una gráfica de draw-down. Cierra con una conclusión clara, sin recomendar comprar ni vender.

Esta consulta comprueba simultáneamente comprensión, selección de métricas, generación de código, control de formato, datos para visualización e interpretación final.

8.5. Tipos de salida y ejemplos visuales

Para que la evaluación sea comprensible, conviene separar qué produce el sistema en cada fase. El resultado final que ve el usuario es una explicación en lenguaje natural, pero antes de llegar a ella el workflow genera artefactos intermedios: un plan analítico, código Python, una salida JSON con métricas y, en consultas de Nivel B o C, datos preparados para representar gráficas. En las ejecuciones de evaluación también se materializaron algunas de esas salidas como imágenes PNG para facilitar la revisión visual y su incorporación a la memoria.

Tipo de salida	Aparece en	Uso en la evaluación
Métricas básicas	Nivel A	Permiten comprobar si el sistema calcula lo esencial de una consulta simple.
Tabla comparativa	Nivel B y C	Resume rentabilidad, volatilidad, drawdown u otras métricas por activo.
Serie normalizada base 100	Nivel B y C	Compara activos con escalas distintas sin confundir precio absoluto con evolución relativa.
Serie de drawdown	Nivel C	Muestra caídas históricas desde máximos previos y complementa la rentabilidad con riesgo observado.
Respuesta interpretada	Todos los niveles	Traduce la salida ejecutada a lenguaje claro, sin añadir datos externos ni recomendaciones.

Tabla 8.3: Tipos de salida generados por el workflow y uso dentro de la evaluación.

La Figura 8.1 muestra un ejemplo de gráfica normalizada para QQQ y SPY durante 2024. Esta visualización es especialmente adecuada cuando se comparan activos con precios de partida diferentes, ya que ambos comienzan en una base común y el lector puede centrarse en la evolución relativa.

La Figura 8.2 representa el drawdown de los mismos activos. En lugar de mostrar sólo el resultado acumulado, esta gráfica permite ver la profundidad de las caídas desde máximos previos y ayuda a explicar por qué una comparación financiera no debe limitarse a la rentabilidad total.

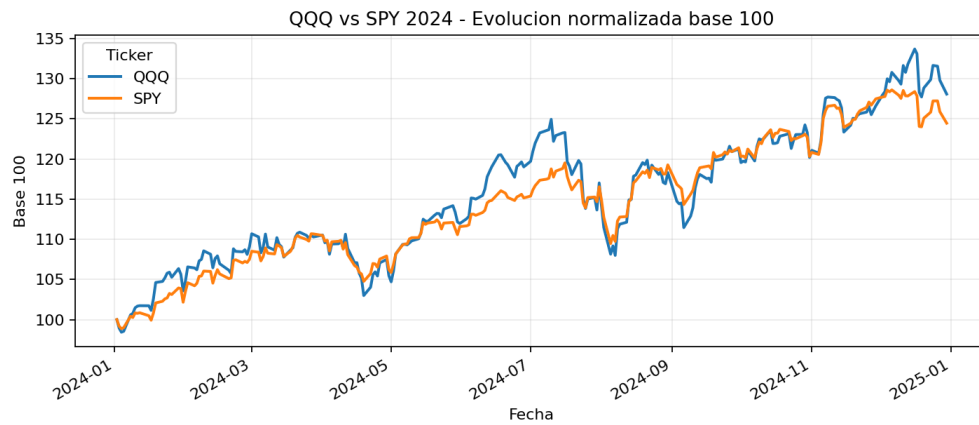


Figura 8.1: Ejemplo de salida visual: evolución normalizada base 100 de QQQ y SPY en 2024.

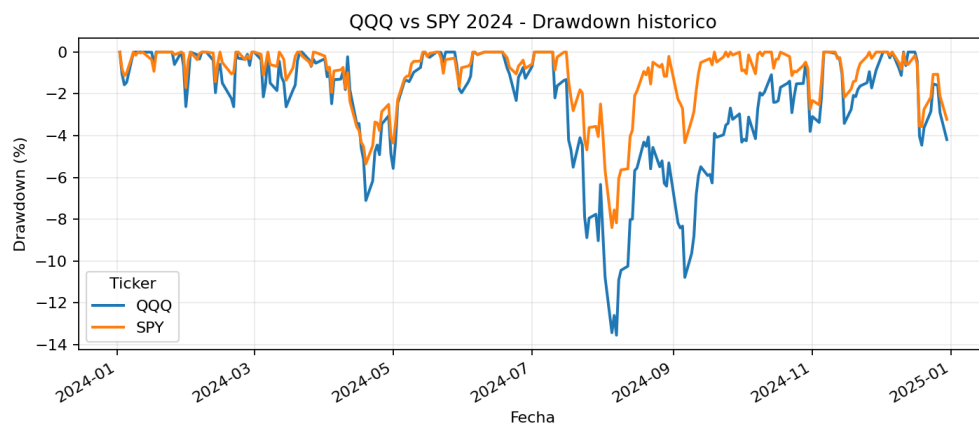


Figura 8.2: Ejemplo de salida visual: drawdown histórico de QQQ y SPY en 2024.

8.6. Resultados observados

La evaluación amplia se ejecutó el 2 de junio de 2026. En total se registraron 45 casos, resultado de cruzar 15 consultas con tres configuraciones LLM. Se considera completado un caso que alcanza la interpretación final con estado `completed`. Los casos parciales también son informativos, porque permiten saber si el modelo llegó a planificar, generar código o ejecutar métricas antes de fallar.

Configuración	Modelo o despliegue	Nivel A	Nivel B	Nivel C	Total
Groq	llama-3.3-70b-versatile	3/5	5/5	5/5	13/15
Gemini	gemini-2.5-flash	4/5	0/5	0/5	4/15
openai/gpt-oss-20b	servido mediante vLLM	4/5	4/5	4/5	12/15

Tabla 8.4: Completitud observada por configuración y nivel en la batería de 15 consultas.

En conjunto, la pasada completa registra 29 casos completados de 45. Groq completa todos los casos B y C, mientras que sus fallos aparecen en consultas A por generación o ejecución de código. Gemini queda condicionado por un límite de cuota 429, por lo que sus fallos posteriores describen disponibilidad de API y no sólo calidad del modelo. `openai/gpt-oss-20b` mantiene una cobertura equilibrada en A/B/C e incluye la query de estrés visual completada.

La lectura más relevante para el TFM no es que un proveedor gane de forma absoluta, sino que el workflow permite observar dónde falla cada ejecución: cuota de proveedor, interpretación inicial, generación de código, validación, ejecución o interpretación final. En las respuestas completadas no se detectaron recomendaciones explícitas de compra o venta.

La Figura 8.3 resume visualmente esta completitud por modelo y nivel. Su función no es cerrar una clasificación estadística definitiva, sino mostrar cómo se materializan los resultados de una pasada exploratoria reproducible.

8.7. Limitaciones metodológicas

La calidad de las respuestas LLM requiere revisión humana y las cifras de latencia dependen de red, cuota y carga del proveedor. Además, una única pasada no permite afirmar robustez estadística. La comparación debe entenderse como exploratoria, trazable y revisable.

El caso de Gemini ilustra una limitación práctica importante: parte de los fallos se producen por cuota del proveedor, no necesariamente por incapacidad del modelo

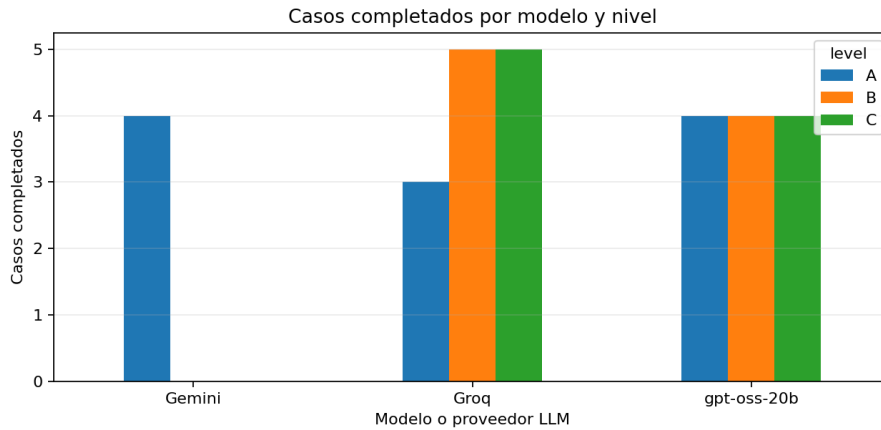


Figura 8.3: Completitud por modelo y nivel en la pasada exploratoria de 15 consultas.

para resolver la tarea. Por ello, la memoria separa disponibilidad de API, errores de formato, fallos de codegen y defectos de interpretación.

El sistema trabaja con datos históricos locales. Su objetivo es describir métricas observadas, no producir asesoramiento financiero, incorporar noticias externas ni predecir el comportamiento futuro de los activos.

8.8. Síntesis

La evaluación combina criterios técnicos y cualitativos. Su aportación principal consiste en revisar por separado interpretación, generación de código, validación, ejecución e interpretación final. Esta estructura permite analizar no sólo si el workflow termina, sino también si responde con datos suficientes, comprensibles y trazables.

Los resultados observados muestran que el MVP es funcional y evaluable, especialmente en consultas B/C con salidas estructuradas y visuales. También muestran que la robustez depende de factores externos, como cuota y disponibilidad de proveedor, y de factores internos, como la estabilidad del código generado y la compactación de salidas largas.

Capítulo 9

Aspectos éticos, seguridad y uso responsable

9.1. Delimitación financiera del sistema

El sistema se orienta exclusivamente al análisis financiero histórico y descriptivo. Sus respuestas deben interpretarse como explicaciones basadas en datos pasados, no como predicciones ni como recomendaciones de inversión. Esta delimitación forma parte del diseño del MVP y debe mantenerse visible tanto en la memoria como en cualquier interfaz futura.

El prototipo calcula métricas como crecimiento, retornos, volatilidad, drawdown e indicadores técnicos básicos. No realiza predicciones, no recomienda comprar o vender activos y no adapta decisiones a perfiles personales de riesgo.

9.2. Riesgos de interpretación

Un riesgo relevante es que el usuario confunda una explicación descriptiva con asesoramiento financiero. Para reducirlo, el sistema debe utilizar formulaciones prudentes, explicitar el período analizado y recordar que los resultados históricos no garantizan comportamientos futuros.

La capa LLM aumenta este riesgo si redacta respuestas demasiado persuasivas. Por ello, la configuración actual pide al modelo interpretar resultados ya calculados, no añadir datos externos y evitar recomendaciones de inversión.

9.3. Seguridad en la generación y ejecución de código

La generación automática de código introduce riesgos específicos. Un script mal generado podría fallar, calcular una métrica incorrecta o acceder a recursos no previs-

tos. En este proyecto, el riesgo se reduce mediante entradas estructuradas, ejecución controlada, validación mínima del contrato y captura explícita de salidas.

El código generado se ejecuta como proceso externo, con registro de salida estándar, salida de error y código de retorno. Esta estrategia permite revisar errores y evita que la respuesta final oculte fallos de ejecución.

9.4. Privacidad y datos

Los datos utilizados son series históricas de mercado y no contienen datos personales. Aun así, las claves de API de proveedores LLM se consideran información sensible y se almacenan exclusivamente en el fichero local `.env`, excluido del control de versiones.

En una versión productiva, sería necesario revisar las condiciones de uso de los proveedores de datos y de modelos, así como las políticas de retención de información enviada a APIs externas.

9.5. Sostenibilidad, costes y recursos

El enfoque incremental reduce costes computacionales. El MVP no entrena modelos desde cero y puede ejecutarse en CPU sobre datos ya descargados. La capa LLM se concentra principalmente en interpretación y permite comparar proveedores sin entrenar ni ajustar pesos.

Esta estrategia también mejora la sostenibilidad del desarrollo: evita ejecuciones innecesarias de modelos grandes, reduce dependencia de infraestructura GPU y permite comparar proveedores antes de elegir una configuración de uso continuado.

9.6. Limitaciones de uso

El sistema no debe utilizarse como herramienta profesional de decisión financiera sin validaciones adicionales. Su valor actual es académico y experimental: demostrar una arquitectura modular, trazable y robusta para automatizar análisis históricos.

Entre las limitaciones principales se encuentran el uso de un conjunto de datos acotado, la dependencia de datos obtenidos desde Yahoo Finance mediante `yfinance` [18], [19], la ausencia de validación financiera profesional y la no incorporación de información fundamental, noticias o variables macroeconómicas.

Capítulo 10

Conclusiones y trabajo futuro

El trabajo desarrollado demuestra la viabilidad de un sistema LLM para análisis financiero histórico basado en una arquitectura modular, trazable y extensible. El MVP valida entradas estructuradas, interpreta la intención del usuario, genera código Python, revisa su seguridad, ejecuta el script sobre CSV locales y redacta una respuesta final a partir de resultados observados. La aportación principal no es predecir el mercado, sino conectar lenguaje natural, cálculo reproducible y explicación financiera prudente.

Los objetivos planteados se cumplen en una versión funcional y evaluable. Se ha definido un workflow modular, se ha incorporado generación de código mediante LLM, se ha añadido una capa de seguridad previa a la ejecución, se han conservado artefactos auditables y se han preparado scripts de evaluación con Groq, Gemini y `openai/gpt-oss-20b` servido mediante vLLM. Además, se ha ampliado la evaluación desde consultas simples hacia una metodología cualitativa por niveles A/B/C, más cercana a la forma real en que un usuario valora una respuesta de análisis.

Una aportación importante es la separación de responsabilidades. El primer agente debe producir código, métricas, tablas o datos de visualización suficientes para responder la consulta. El segundo agente debe interpretar esos resultados con claridad, distinguir dato histórico, lectura y limitaciones, y evitar recomendaciones de inversión. Esta división permite revisar el sistema por partes y detectar si un fallo procede de la comprensión de la query, del código, de la ejecución o de la interpretación final.

La evaluación realizada muestra un estado prometedor, pero todavía experimental. En la pasada amplia documentada el 2 de junio de 2026 se ejecutaron 45 casos, al cruzar 15 consultas A/B/C con tres configuraciones LLM. Groq con `llama-3.3-70b-versatile` [25] completó 13 de 15 casos; `openai/gpt-oss-20b` [27] servido mediante vLLM completó 12 de 15, incluyendo la query de estrés visual; y Gemini con `gemini-2.5-flash` [26] completó 4 de 15, con la pasada condicionada por límites de cuota del proveedor. Por tanto, no se afirma una superioridad objetiva definitiva de ningún proveedor; se presenta evidencia observada bajo condiciones concretas.

Los resultados también han permitido mejorar el diseño. Las consultas profesionales o visuales generaban salidas muy extensas, especialmente cuando incluían series temporales completas para gráficas. Esto reveló un problema práctico de contexto para el segundo agente. La solución incorporada compacta la información enviada al LLM interpretador, manteniendo la salida completa como artefacto auditable. Este ajuste refuerza la robustez del MVP sin perder trazabilidad.

El prototipo trabaja con un conjunto acotado de datos históricos ya descargados. No aborda predicción futura, recomendación de inversión, análisis fundamental ni incorporación de noticias. La generación de código mediante LLM queda limitada por contratos, validaciones y revisión posterior, por lo que requiere más pruebas antes de cualquier uso fuera del entorno académico. La revisión cualitativa también incorpora juicio humano, aunque se controla mediante criterios explícitos y una plantilla común.

Como trabajo futuro se proponen varias líneas: repetir la batería A/B/C en varias rondas para reducir el efecto de cuotas y disponibilidad puntual, ampliar las consultas de estrés, reforzar el análisis de seguridad del código generado, registrar experimentos con herramientas MLOps, automatizar parte de la rúbrica cualitativa sin sustituir la revisión humana y mejorar la generación de visualizaciones exportables. También sería natural incorporar pruebas con más activos, ventanas temporales y perfiles de usuario, manteniendo siempre la separación entre rendimiento histórico observado e interpretación.

En conjunto, el TFM se encuentra en un punto sólido para memoria: existe un MVP ejecutable, una metodología defendible, evidencias guardadas, comparación exploratoria entre modelos y ejemplos visuales que muestran cómo se ha aumentado progresivamente la dificultad de las consultas.

Referencias

- [1] A. Vaswani et al., «Attention Is All You Need,» en *Advances in Neural Information Processing Systems*, 2017. dirección: <https://arxiv.org/abs/1706.03762>
- [2] Analytics Vidhya, *60+ Generative AI Terms You Must Know*, 2024. visitado 25 de mayo de 2026. dirección: <https://www.analyticsvidhya.com/blog/2024/01/generative-ai-terms/>
- [3] D. Araci, «FinBERT: Financial Sentiment Analysis with Pre-trained Language Models,» *arXiv preprint arXiv:1908.10063*, 2019. DOI: 10.48550/arXiv.1908.10063 arXiv: 1908.10063 [cs.CL]. dirección: <https://arxiv.org/abs/1908.10063>
- [4] S. Wu et al., «BloombergGPT: A Large Language Model for Finance,» *arXiv preprint arXiv:2303.17564*, 2023. DOI: 10.48550/arXiv.2303.17564 arXiv: 2303.17564 [cs.LG]. dirección: <https://arxiv.org/abs/2303.17564>
- [5] H. Yang, X.-Y. Liu y C. D. Wang, «FinGPT: Open-Source Financial Large Language Models,» *arXiv preprint arXiv:2306.06031*, 2023. DOI: 10.48550/arXiv.2306.06031 arXiv: 2306.06031 [q-fin.CP]. dirección: <https://arxiv.org/abs/2306.06031>
- [6] T. Zhou, P. Wang, Y. Wu y H. Yang, «FinRobot: AI Agent for Equity Research and Valuation with Large Language Models,» *arXiv preprint arXiv:2411.08804*, 2024. DOI: 10.48550/arXiv.2411.08804 arXiv: 2411.08804 [q-fin.CP]. dirección: <https://arxiv.org/abs/2411.08804>
- [7] Y. Xiao, E. Sun, D. Luo y W. Wang, «TradingAgents: Multi-Agents LLM Financial Trading Framework,» *arXiv preprint arXiv:2412.20138*, 2024. DOI: 10.48550/arXiv.2412.20138 arXiv: 2412.20138 [q-fin.TR]. dirección: <https://arxiv.org/abs/2412.20138>
- [8] S. Yao et al., «ReAct: Synergizing Reasoning and Acting in Language Models,» en *International Conference on Learning Representations*, 2023. arXiv: 2210.03629 [cs.CL]. dirección: <https://arxiv.org/abs/2210.03629>

- [9] T. Schick et al., «Toolformer: Language Models Can Teach Themselves to Use Tools,» *arXiv preprint arXiv:2302.04761*, 2023. DOI: 10.48550/arXiv.2302.04761 arXiv: 2302.04761 [cs.CL]. dirección: <https://arxiv.org/abs/2302.04761>
- [10] L. Wang et al., «A Survey on Large Language Model based Autonomous Agents,» *arXiv preprint arXiv:2308.11432*, 2023. DOI: 10.48550/arXiv.2308.11432 arXiv: 2308.11432 [cs.AI]. dirección: <https://arxiv.org/abs/2308.11432>
- [11] M. Chen et al., «Evaluating Large Language Models Trained on Code,» *arXiv preprint arXiv:2107.03374*, 2021. DOI: 10.48550/arXiv.2107.03374 arXiv: 2107.03374 [cs.LG]. dirección: <https://arxiv.org/abs/2107.03374>
- [12] Y. Lai et al., «DS-1000: A Natural and Reliable Benchmark for Data Science Code Generation,» *arXiv preprint arXiv:2211.11501*, 2022. DOI: 10.48550/arXiv.2211.11501 arXiv: 2211.11501 [cs.SE]. dirección: <https://arxiv.org/abs/2211.11501>
- [13] J.-B. Alayrac et al., «Flamingo: A Visual Language Model for Few-Shot Learning,» en *Advances in Neural Information Processing Systems*, 2022. dirección: <https://arxiv.org/abs/2204.14198>
- [14] OpenAI, *GPT-4V(ision) System Card*, 2023. visitado 3 de jun. de 2026. dirección: <https://openai.com/index/gpt-4v-system-card/>
- [15] Gemini Team, Google, «Gemini 2.5: Pushing the Frontier with Advanced Reasoning, Multimodality, Long Context, and Next Generation Agentic Capabilities,» Google, inf. téc., 2025. dirección: https://storage.googleapis.com/deepmind-media/gemini/gemini_v2_5_report.pdf
- [16] Z. Ji et al., «Survey of Hallucination in Natural Language Generation,» *ACM Computing Surveys*, 2022. DOI: 10.1145/3571730 arXiv: 2202.03629 [cs.CL]. dirección: <https://arxiv.org/abs/2202.03629>
- [17] P. Lewis et al., «Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks,» en *Advances in Neural Information Processing Systems*, 2020. arXiv: 2005.11401 [cs.CL]. dirección: <https://arxiv.org/abs/2005.11401>
- [18] R. Aroussi, *yfinance Python Library*, 2026. visitado 25 de mayo de 2026. dirección: <https://github.com/ranaroussi/yfinance>
- [19] Yahoo Finance Help, *Download historical data in Yahoo Finance*, 2026. visitado 25 de mayo de 2026. dirección: <https://help.yahoo.com/kb/finance/historical-prices-adjusted-close-sln2311.html>

- [20] U.S. Securities and Exchange Commission, *Investor Bulletin: Performance Claims*, 2022. visitado 27 de mayo de 2026. dirección: <https://www.investor.gov/introduction-investing/general-resources/news-alerts/alerts-bulletins/investor-bulletins-47>
- [21] Investor.gov, *What is Risk?* 2026. visitado 27 de mayo de 2026. dirección: <https://www.investor.gov/introduction-investing/investing-basics/what-risk>
- [22] CFA Institute, *Portfolio Risk and Return: Part I*, 2026. visitado 27 de mayo de 2026. dirección: <https://www.cfainstitute.org/insights/professional-learning/refresher-readings/2026/portfolio-risk-return-part-1>
- [23] CFA Institute, *Performance Drawdowns in Asset Management: Extending Draw-down Analysis to Active Returns*, 2018. visitado 27 de mayo de 2026. dirección: <https://rpc.cfainstitute.org/research/cfa-digest/2018/01/dig-v48-n1-3>
- [24] Atlassian, *A Complete Guide to Line Charts*, 2026. visitado 27 de mayo de 2026. dirección: <https://www.atlassian.com/data/charts/line-chart-complete-guide>
- [25] Groq, *Llama 3.3 70B Model Documentation*, 2026. dirección: <https://console.groq.com/docs/model/llama-3.3-70b-versatile>
- [26] Google Cloud, *Gemini 2.5 Flash Model Documentation*, 2025. dirección: <https://docs.cloud.google.com/vertex-ai/generative-ai/docs/models/gemini/2-5-flash>
- [27] OpenAI, *gpt-oss-120b & gpt-oss-20b Model Card*, 2025. DOI: 10.48550/arXiv.2508.10925 arXiv: 2508.10925 [cs.CL]. dirección: <https://arxiv.org/abs/2508.10925>
- [28] LangChain, «LangGraph: Building Stateful, Multi-Actor Applications with LLMs,» 2024. dirección: <https://langchain-ai.github.io/langgraph/>
- [29] Groq, *Groq API Documentation*, 2026. dirección: <https://console.groq.com/docs>
- [30] Meta, *Llama 3.3 70B Instruct Model Card*, 2024. dirección: https://github.com/meta-llama/llama-models/blob/main/models/llama3_3/MODEL_CARD.md
- [31] AI@Meta, «The Llama 3 Herd of Models,» *arXiv preprint arXiv:2407.21783*, 2024. DOI: 10.48550/arXiv.2407.21783 arXiv: 2407.21783 [cs.AI]. dirección: <https://arxiv.org/abs/2407.21783>

- [32] Google AI for Developers, *Gemini API Documentation*, 2026. dirección: <https://ai.google.dev/gemini-api/docs>
- [33] OpenAI, *OpenAI o1 System Card*, 2024. DOI: 10.48550/arXiv.2412.16720 arXiv: 2412.16720 [cs.AI]. dirección: <https://arxiv.org/abs/2412.16720>
- [34] DeepSeek-AI, «DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning,» *arXiv preprint arXiv:2501.12948*, 2025. DOI: 10.48550/arXiv.2501.12948 arXiv: 2501.12948 [cs.CL]. dirección: <https://arxiv.org/abs/2501.12948>

Apéndice A

Anexos

A.1. Ejecución del MVP

El proyecto puede ejecutarse con perfiles LLM mediante los ejemplos incluidos en el repositorio:

```
python run_mvp.py --example growth_nvda
python run_mvp.py --example compare_nvda_amd
python run_mvp.py --example overview_aapl
python run_mvp.py --example returns_qqq_spy
python run_mvp.py --example risk_qqq_spy
python run_mvp.py --example technical_aapl
```

Además de estos ejemplos funcionales, se prepararon scripts específicos para la evaluación cualitativa, la demostración para cliente y la comparación progresiva entre proveedores LLM:

```
python scripts\generate_qualitative_client_demo.py
python scripts\generate_qualitative_demo_report.py
python scripts\evaluate_progressive_llm_scope.py ^
    --profiles groq gemini vllm
```

La última orden puede restringirse a una muestra concreta de dificultad creciente:

```
python scripts\evaluate_progressive_llm_scope.py ^
    --profiles groq gemini vllm ^
    --cases level_a_nvda_growth level_a_qqq_spy_returns ^
        level_b_qqq_spy_clear_compare ^
        level_c_qqq_spy_professional ^
        stress_visual_qqq_spy_monthly
```


A.2. Configuración de APIs

Las claves de API se guardan localmente en el fichero `.env`, que no debe subirse al repositorio. El proyecto asume llamadas LLM en el flujo principal, por lo que no hay variables de activación.

Para activar Groq con `llama-3.3-70b-versatile` [25], modelo Llama 3.3 70B Instruct orientado a razonamiento general y baja latencia, la configuración es:

```
llama-3.3-70b-versatile [25]
```

```
LLM_PROFILE=groq  
GROQ_MODEL=llama-3.3-70b-versatile
```

Para activar Gemini con `gemini-2.5-flash` [26], modelo Flash de Gemini 2.5 orientado a rapidez y razonamiento, la configuración es:

```
gemini-2.5-flash [26]
```

```
LLM_PROFILE=gemini  
GEMINI_MODEL=gemini-2.5-flash
```

Para activar `openai/gpt-oss-20b` [27] servido mediante vLLM, modelo abierto de la familia gpt-oss usado por su capacidad de razonamiento, seguimiento de instrucciones y generación de código, se define un perfil técnico local para el despliegue compatible con Chat Completions:

```
openai/gpt-oss-20b [27]
```

```
LLM_PROFILE=vllm  
VLLM_BASE_URL=https://servidor-vllm.example/v1  
VLLM_MODEL=openai/gpt-oss-20b
```

A.3. Batería de consultas A/B/C

La batería cualitativa se diseñó para comprobar que el sistema no sólo termina la ejecución, sino que adapta la profundidad del análisis a la consulta. La versión utilizada en la evaluación amplia contiene 15 consultas equilibradas: cinco de Nivel A, cinco de Nivel B y cinco de Nivel C. La definición ejecutable se conserva en `scripts/generate_qualitative_demo_report.py` y la descripción completa en `docs/bateria_evaluacion_15_queries.md`.

Tabla A.1: Batería cualitativa de 15 consultas por nivel de profundidad.

Nivel	Consulta resumida	Objetivo de evaluación
A	Crecimiento de Nvidia en los últimos cinco años.	Crecimiento histórico básico.
A	Comparación simple de Nvidia y AMD en dos años.	Comparación por rentabilidad.
A	Visión general de AAPL en tres meses.	Resumen descriptivo.
A	Retornos de QQQ y SPY en 2024.	Retornos históricos resumidos.
A	Riesgo histórico de QQQ y SPY en 2024.	Riesgo, volatilidad y extremos.
B	Análisis completo de Nvidia con tabla y gráfica de evolución.	Tabla y serie de precio.
B	Comparación clara de QQQ y SPY con tabla y gráfica normalizada.	Comparación relativa base 100.
B	Retorno y riesgo de AAPL con explicación sencilla.	Relación retorno-riesgo.
B	Bitcoin 2024 con rentabilidad, volatilidad, drawdown y evolución.	Cobertura de criptoactivo.
B	EUR/USD intradía con variación, rango, volatilidad y evolución.	Cobertura de divisa horaria.
C	Informe profesional QQQ/SPY con tabla, normalización y drawdown.	Informe comparativo completo.
C	Ranking NVDA/AMD por rentabilidad, riesgo y drawdown.	Comparación multicriterio.
C	Informe detallado del S&P 500 desde 2020.	Serie larga de índice amplio.
C	Informe horario del oro con tabla, evolución, volatilidad y rango.	Materia prima e intervalo horario.
C	QQQ/SPY 2024 con cuatro bloques obligatorios de tabla y visualización.	Estrés de formato visual.

A.4. Query de estrés visual

Para forzar al sistema a respetar una estructura concreta se incorporó una query adicional de estrés visual:

Compara QQQ y SPY durante 2024 como si fuera un informe para un cliente no técnico. Quiero que me muestres los datos exactamente en cuatro bloques: 1) una tabla resumen con rentabilidad total, volatilidad anualizada,

máximo drawdown, mejor mes y peor mes; 2) una tabla de clasificación mensual indicando qué activo ganó cada mes; 3) datos para una gráfica normalizada base 100 de ambos activos; 4) datos para una gráfica de draw-down. Cierra con una conclusión clara, sin recomendar comprar ni vender.

Esta consulta se eligió porque comprueba simultáneamente comprensión, selección de métricas, generación de código, control de formato, datos para visualización e interpretación final sin recomendación de inversión.

A.5. Prompts usados por los agentes

Los prompts del sistema se concentran en `src/llm/prompts.py`. En lugar de usar un único prompt conversacional, el workflow define instrucciones diferentes para cada bloque. Todos comparten una restricción de dominio: trabajar sólo con datos históricos ya cargados en CSV, sin predicción futura, sin recomendación de inversión y sin análisis fundamental complejo.

A.5.1. Prompt base del sistema

```
Eres un modulo LLM analitico financiero dentro de un TFM.  
Trabajas solo con datos historicos ya cargados en CSV.  
No haces prediccion futura, recomendacion de inversion  
ni analisis fundamental complejo.
```

A.5.2. Agente 1: interpretación de consulta

El primer prompt convierte la solicitud en un plan auditable. No pide una respuesta final, sino un JSON con intención, tipo de análisis, métricas, columnas necesarias, requisitos de datos, requisitos de salida y razonamiento.

```
Devuelve solo JSON valido. Interpreta la intencionalidad  
de la consulta, decide que analisis financiero hace falta  
y describe que debe calcular el script.  
Clasifica implicitamente la profundidad como Nivel A, B o C.  
No inventes datos externos: usa solo los CSV, tickers y fechas.
```

A.5.3. Agente 1: generación de código

La segunda llamada del primer agente materializa el plan en código Python. Su salida vuelve a estar estructurada: un JSON con un único campo `code`. El contrato impide que el modelo responda al usuario en esta fase.

Devuelve solo JSON valido con un campo code.
El codigo debe ser Python ejecutable, leer el payload recibido por argv[1] y escribir JSON en stdout.
No redactes la respuesta final al usuario: calcula metricas y devuelve resultados estructurados.
Si se solicita una grafica, representa sus datos en JSON; no crees ficheros de imagen y limita cada serie visual.

A.5.4. Reparación de código

Cuando el script falla o no cumple el contrato, el prompt de reparación recibe el código previo y el error observado. Esta llamada no relaja la seguridad: la versión corregida vuelve a validarse antes de ejecutarse.

El codigo generado fallo o fue rechazado.
Devuelve solo JSON valido con un campo code que contenga una version corregida completa del script Python.
Manten el objetivo analitico original, incluido el nivel A/B/C.
No incluyas markdown ni explicaciones.

A.5.5. Agente 2: interpretación final

El segundo agente recibe la salida ejecutada y el plan. Su tarea es explicar, no recalcular. Por ello el prompt insiste en fidelidad a las métricas recibidas y en separar datos históricos, interpretación y limitaciones.

Redacta una respuesta breve en espanol para usuario final.
Interpreta las metricas recibidas y explica que significan en lenguaje claro.
Distingue datos historicos observados, interpretacion y limitaciones.
Respeta las metricas recibidas, no anadas datos externos, no hagas predicciones y evita recomendaciones de inversion.

A.6. Ejecuciones completas resumidas por nivel

Las siguientes tablas resumen tres ejecuciones reales realizadas con el modelo gpt-oss-20b, identificado en configuración como `openai/gpt-oss-20b` y servido mediante vLLM. Se selecciona un ejemplo de Nivel A, uno de Nivel B y uno de Nivel C para mostrar qué entra al primer agente, qué plan genera, qué código se ejecuta, qué salida

produce y qué recibe el segundo agente para redactar la respuesta final. Los ficheros completos quedan conservados en **results/**.

Tabla A.2: Resumen de ejecuciones A/B/C con `openai/gpt-oss-20b`.

Nivel	Entrada al flujo	Salida del agente 1	Salida ejecutada e interpretación
A	Consulta: “Cuánto ha crecido Nvidia en los últimos 5 años”. Ticker: NVDA. Periodo: 5y. CSV: <code>data/raw/cunto_ha_crecido_nvidia_en_5_aos.csv</code> .	Plan: <code>price_growth_analysis</code> . Métricas: crecimiento a 5 años y CAGR. Sin tabla ni gráfica obligatoria. Código: <code>results/code/generated_20260602_171109_113418.py</code> .	Ejecución: crecimiento 1273,33 % y CAGR 68,87 %. El agente 2 explica la magnitud del crecimiento y aclara que el CAGR suaviza la evolución histórica, sin convertirlo en predicción.
B	Consulta: comparar QQQ y SPY en 2024 con tabla y gráfica normalizada. Tickers: QQQ, SPY. Fechas: 2024-01-01 a 2024-12-31. CSV: <code>data/raw/qqq_y_spy_desde_20240101_hasta_20241231.csv</code> .	Plan: <code>comparative_performance</code> . Métricas: retorno diario, retorno acumulado, volatilidad, máximo drawdown y Sharpe. Requisitos: tabla comparativa y serie normalizada. Código: <code>results/code/generated_20260602_172315_052061.py</code> .	Ejecución: QQQ obtiene retorno acumulado 28,07 %, volatilidad 17,96 % y drawdown -16,94 %; SPY obtiene retorno 24,45 %, volatilidad 12,61 % y drawdown -10,05 %. El agente 2 traduce la comparación a una lectura clara de retorno frente a riesgo.
C	Query de estrés visual QQQ vs SPY 2024 con cuatro bloques: tabla resumen, ranking mensual, serie normalizada y drawdown. Tickers: QQQ, SPY. CSV diario de 2024.	Plan: <code>Comparative performance analysis</code> . Métricas: retorno total, volatilidad anualizada, drawdown, mejor/peor mes, ganador mensual, serie normalizada y serie de drawdown. Código: <code>results/code/generated_20260602_173942_390008.py</code> .	Ejecución: genera tabla por activo, 11 filas de ranking mensual, 53 puntos semanales para serie normalizada y 53 puntos de drawdown. El agente 2 redacta un informe para usuario no técnico y cierra sin recomendar comprar ni vender.

A.6.1. Fragmentos de código generado

En el Nivel A el script generado se limita al cálculo directo de crecimiento y CAGR:

```
first_close = float(close.iloc[0][ticker])
last_close = float(close.iloc[-1][ticker])
growth = (last_close - first_close) / first_close * 100
cagr = ((last_close / first_close) ** (1/5) - 1) * 100
```

En el Nivel B aparece ya una función auxiliar para calcular métricas comparables y datos de visualización normalizados:

```
daily_ret = close_df.pct_change().dropna()
cum_ret = (1 + daily_ret).cumprod() - 1
vol = ret.std() * np.sqrt(252)
max_dd = (cum - cum.cummax()).min()
norm_cum = 100 * (1 + cum_ret)
```

En el Nivel C el código amplía la salida con ranking mensual, serie normalizada y serie de drawdown:

```
monthly_ret_all = close.resample('M').ffill().pct_change().dropna()
ranking.append({'month': month.strftime('%Y-%m'), 'winner': winner})
norm_week = norm.resample('W').last()
drawdown = (close / close.cummax()) - 1
dd_week = drawdown.resample('W').last()
```

A.6.2. Artefactos de trazabilidad

Cada ejecución conserva los mismos tipos de fichero. El **payload** recoge lo que entra al script, incluyendo la consulta, tickers, rutas CSV y plan del agente 1. El script Python es la salida generada por el primer agente. El fichero **stdout** contiene el JSON ejecutado que se envía al segundo agente, y **stderr** permite diagnosticar errores de ejecución si los hubiera.

Nivel	Artefactos principales
A	results/logs/payload_20260602_171109_113418.json, results/code/generated_20260602_171109_113418.py, results/logs/stdout_20260602_171109_113418.json.
B	results/logs/payload_20260602_172315_052061.json, results/code/generated_20260602_172315_052061.py, results/logs/stdout_20260602_172315_052061.json.
C	results/logs/payload_20260602_173942_390008.json, results/code/generated_20260602_173942_390008.py, results/logs/stdout_20260602_173942_390008.json.

Tabla A.3: Artefactos reales asociados a los ejemplos de ejecución A/B/C.

A.7. Documentación estable

La documentación Markdown estable conserva el diseño, la metodología y los aprendizajes reutilizables. La pasada amplia seleccionada para la memoria se conserva como evidencia en **results/reports/evaluacion_15_queries_llms.md**; otros informes de depuración se tratan como artefactos temporales que pueden regenerarse si se repite la evaluación:

- **docs/evolucion_mvp_tfm.md**: evolución incremental del MVP desde dos intencionalidades hasta el flujo actual con codegen controlado.

- `docs/arquitectura_llm_codegen.md`: descripción técnica del flujo LLM, nodos, prompts, seguridad y ejecución.
- `docs/llm_api_setup.md`: configuración de Groq, Gemini y `openai/gpt-oss-20b` servido mediante vLLM compatible con Chat Completions.
- `docs/planificacion_evaluacion_tfm.md`: protocolo de evaluación, familias de consultas, aprendizajes técnicos y líneas de trabajo futuro.
- `docs/control_tamano_salidas_visualizacion.md`: justificación de la compactación de salidas visuales largas antes de la interpretación final.
- `docs/evaluacion_cualitativa_llm.md`: metodología cualitativa, criterios, niveles A/B/C y rúbrica.

A.8. Plantilla de revisión manual

La siguiente plantilla se usa para revisar cada ejecución sin presentar la evaluación como una métrica matemática absoluta:

Criterio	Valoración	Observaciones
Comprensión de la consulta	Excelente / Correcto / Parcial / Deficiente	
Selección de métricas y datos	Excelente / Correcto / Parcial / Deficiente	
Código generado	Excelente / Correcto / Parcial / Deficiente	
Tablas y gráficas solicitadas	Excelente / Correcto / Parcial / Deficiente	
Exactitud frente a la salida ejecutada	Excelente / Correcto / Parcial / Deficiente	
Claridad del análisis final	Excelente / Correcto / Parcial / Deficiente	
Utilidad para el usuario	Excelente / Correcto / Parcial / Deficiente	
Reconocimiento de limitaciones	Excelente / Correcto / Parcial / Deficiente	
Ausencia de recomendación de inversión	Excelente / Correcto / Parcial / Deficiente	
Trazabilidad consulta-código-resultados	Excelente / Correcto / Parcial / Deficiente	

Tabla A.4: Plantilla de revisión manual cualitativa.

A.9. Documentación interna de apoyo

Durante el desarrollo se conservaron notas técnicas y documentos metodológicos en Markdown. Estos documentos no sustituyen a la memoria, pero permiten reconstruir decisiones de diseño, documentar la evaluación final y registrar limitaciones conocidas sin mezclar esos contenidos con salidas provisionales.

Documento interno	Uso principal en la memoria
docs/evolucion_mvp_tfm.md	Motivación y metodología incremental del prototipo.
docs/arquitectura_llm_codegen.md	Diseño del sistema, nodos, contrato de codegen y trazabilidad.
docs/control_tamano_salidas_visualizacion.md	Compactación de salidas largas y gestión de contexto en Nivel B/C.
docs/evaluacion_cualitativa_llm.md	Criterios de buen análisis, niveles A/B/C y plantilla de revisión.
docs/planificacion_evaluacion_tfm.md	Protocolo definitivo, aprendizajes técnicos, familias de consultas y trabajo futuro.

Tabla A.5: Relación entre documentación Markdown y secciones de la memoria.